◆ DRAGONAI · 烛龙智元

AI FDE 工程师就业实训营

学生讲义

Day 5

Git 团队协作与全栈应用部署

从 Git 小团队协作到宝塔，Docker，域名与 HTTPS

杭州烛龙智元科技有限公司

目录

学习目标 1

01 开场与课程主线 1

1.1 课程地图与主线 1

1.2 演示项目与部署结构 2

02 Git 核心模型与本地工作流 3

2.1 快照与 Commit 3

2.2 Git 的区域划分 4

2.3 分支的概念 5

2.4 身份配置与获取仓库 6

2.5 最小提交循环 6

2.6 Commit 与 Push 8

2.7 阅读提交历史 8

2.8 分支的日常命令 9

2.9 撤销操作的选择 10

03 GitHub 小团队协作流程 10

3.1 小团队开发的完整流程 10

3.2 开发者的日常命令流 11

3.3 合并前快速检查 12

3.4 开发期间同步 main 12

3.5 密钥的存放与泄漏处理 12

04 冲突处理与 Review 及主分支保护 13

4.1 冲突标记的读法 13

4.2 命令行解决冲突的标准步骤 14

4.3 GitHub 在线解决简单行冲突 15

4.4 减少冲突的做法 15

4.5 Rebase 与历史重写 15

4.6 主分支保护规则 16

05 服务器购买与部署架构 17

5.1 购买服务器 17

5.2 端口开放策略 18

5.3 宝塔与 Docker 的分工 18

06 宝塔部署的关键知识点 19

6.1 环境组件与职责 19

6.2 静态前端与 SPA 路由 20

6.3 FastAPI 运行模型与进程守护 20

6.4 Nginx 反向代理与面板字段 21

6.5 前后端联调的验证顺序 22

6.6 Node 后端与项目配置 22

6.7 服务器上的环境变量 24

07 Docker 基础与部署模型 25

7.1 Docker 核心部件的关系 25
7.2 最小 Dockerfile 读法 26
7.3 Docker 层与缓存 27
7.4 构建运行与端口映射 27
7.5 容器排错命令 28
7.6 用服务名互访 29
7.7 Compose 管理多容器应用 29
7.8 用 Claude Code 准备部署 29
08 域名与反向代理及 SSL 证书 30
8.1 A 记录与 CNAME 记录 31
8.2 用 dig 验证 DNS 31
8.3 单个域名承载前后端 31
8.4 代理头与流式输出 32
8.5 ACME 签发与验证方式 33
8.6 宝塔申请证书与 HTTPS 跳转 33
8.7 Mixed Content 与 CORS 排错 34
8.8 上线验证与证书续签 35
09 运维排错回滚与课堂实战 36
9.1 故障分层定位与日志 36
9.2 常见错误码排查 37
9.3 数据库连接与资源不足 37

9.4 备份恢复与安全基线 38
9.5 规范发布与回滚条件 39
9.6 手工部署与 CI/CD 40
9.7 课堂综合实战 40
9.8 官方参考资料 40
练习与作业 **41**
复习题 41
课后作业 42
A 拓展：Qwen3.6 本地模型部署 **42**
A.1 路线选择与内存需求 42
A.2 安装并启动 Studio 43
A.3 模型搜索与量化选择 43
A.4 推理模式与参数 44
A.5 Studio 中的功能验证 44
A.6 llama.cpp 命令行路线 45
A.7 llama-server 兼容接口 46
A.8 MTP 可选加速 46

学习目标

第五天沿 Git 团队协作与全栈应用部署两条主线展开：先用 Git 完成小团队协作开发，再用宝塔或 Docker 部署应用并配置域名与 HTTPS，最终把 Day 3 与 Day 4 的项目交付给另一个人访问。

1. 说明 Git 用快照，Commit 与分支记录项目历史的方式，区分工作区，暂存区与本地仓库。
2. 完成一轮小团队协作：在任务分支上小步提交并推送，经同伴检查后合并，遇到冲突时按标准步骤解决。
3. 区分 Merge 与 Rebase 对历史的不同处理，说明主分支保护规则怎样让 `main` 始终代表可部署版本。
4. 区分宝塔部署与 Docker 部署的分工，说明 Nginx，前端构建产物与后端进程守护在部署中各自承担的工作。
5. 完成域名，`/api` 反向代理与 SSL 证书的配置，让另一个人用手机网络访问并完成关键流程测试。
6. 完成一次故障与回滚演练：主动制造端口错误，按日志定位并恢复，再更新一个版本，然后回滚到上一个版本。

01
开场与课程主线

第五天的内容覆盖一个全栈项目从团队协作到公网上线的全过程，包括 Git 协作，服务器部署以及域名与 HTTPS。

1.1 课程地图与主线

第五天的内容分为四个模块。

模块	内容
Git 协作	分支与 Commit，Push 与冲突处理，主分支保护
宝塔部署	服务器与 Nginx，前端构建，FastAPI 与 Node 的进程守护
Docker	Dockerfile 与 Compose，附录另讲 Qwen3.6 本地模型部署
公网运维	DNS 与域名，反向代理与 SSL，日志与故障定位

四个模块归入两条主线。主线 A 是 Git 协作，包括四项做法：

- 用分支隔离任务。

- 用 Commit 记录可解释的变化。
- 用 Push 共享分支，同伴检查后合并。
- 用规则保护主分支。

主线 B 是公网部署，由四个环节配合完成：

- 服务器持续运行进程。
- Nginx 接收公网请求。
- 宝塔或 Docker 管理应用。
- 域名与 SSL 提供可信的访问入口。

两条主线共用一个贯穿项目：前端采用 Node 技术栈，后端采用 Python FastAPI，数据库采用 MySQL。后端也可以换成 Node 实现。

1.2 演示项目与部署结构

演示项目各部分的技术选择与约定如表所示。

部分	技术与约定
前端	Vite，React 或 Vue 等 Node 技术栈
包管理	pnpm 与 `pnpm-lock.yaml`
后端	FastAPI，uv 与 `uv.lock`
数据	MySQL，生产环境不暴露公网端口
入口	/ 访问前端，`/api` 访问后端

图中自上而下是部署后一次访问经过的路径。浏览器访问 `app.example.com`，请求先到达 Nginx。Nginx 承担两项工作：一是直接返回前端的静态文件，二是**反向代理**，即把请求转交给后面的服务，图中简写为“反代”。Nginx 按路径分流，`/` 交给前端，`/api` 交给 FastAPI。

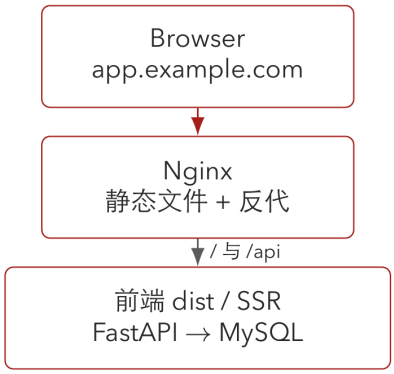


图 1　一次访问依次经过浏览器，Nginx 与后面的服务。

前端可以是构建出的 dist 静态文件，也可以是在服务端生成页面的 SSR 服务。FastAPI 处理请求时再读写 MySQL。前端与后端共用同一个域名，靠路径区分，MySQL 只在内部由后端访问。

表中的 `pnpm-lock.yaml` 与 `uv.lock` 是**锁文件**，记录项目依赖的确切版本。前端用 pnpm 安装依赖，后端用 uv 安装依赖，两者都以锁文件为准，部署出来的环境才能与开发环境一致。

小结

Git 协作与公网部署是第五天的两条主线，前者管理多人开发的代码与历史，后者让应用稳定地对外服务。演示项目的前端，后端与数据库各有确定的技术选择，依赖版本由锁文件固定。部署后的访问统一从 Nginx 进入，再按路径到达前端或 FastAPI，数据库不对外暴露。

02
Git 核心模型与本地工作流

学习 Git 时，先理解快照，Commit 与分支三个概念，再学习具体命令。

2.1 快照与 Commit

Git 用一串项目**快照**记录历史。每次 Commit 都指向提交时项目的完整快照。快照虽然完整，没有变化的文件却不会重复保存，Git 直接复用已有的对象。

图中横轴是时间，每一列是一个版本，竖向列出各个文件在这个版本中的状态。实线框表示文件在这一版有变化，保存了新内容。虚线框表示文件没有变化，复用之前已有的对象。

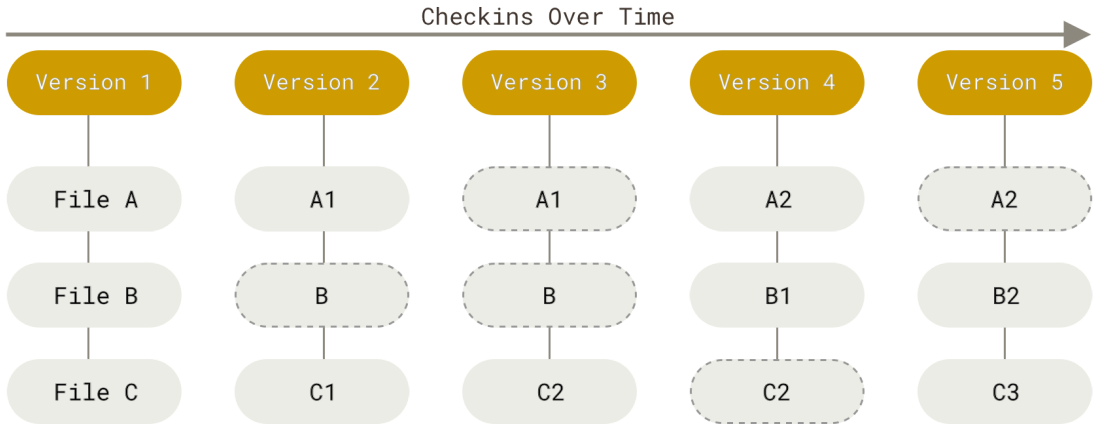


图 2　Git 按时间保存一串快照，不必为每个版本复制整个项目。来源：Pro Git。

一个 Commit 由五项信息组成。

组成	作用
快照	提交时项目的完整状态
作者	记录由谁提交
时间	记录何时提交
说明	记录为什么改，改了什么
父 Commit	指向上一个 Commit，把快照连成历史链

Commit 串联起来，就形成可追踪，可比较的项目历史。借助作者，时间与说明，团队可以随时回答“谁在什么时候，为什么改了什么”。

每个 Commit 用一个 ID 标识，不同 Commit 的 ID 互不相同，例如 `0fd229a`。Commit 不直接列出文件，而是指向一个 Tree，由 Tree 描述目录和其中的文件。图中灰色方框是 Commit，记录了所指向的 Tree，作者与提交说明。中间的青色方框是 Tree，列出目录里的每个文件。右边三个黄色方框是这些文件各自的内容对象。

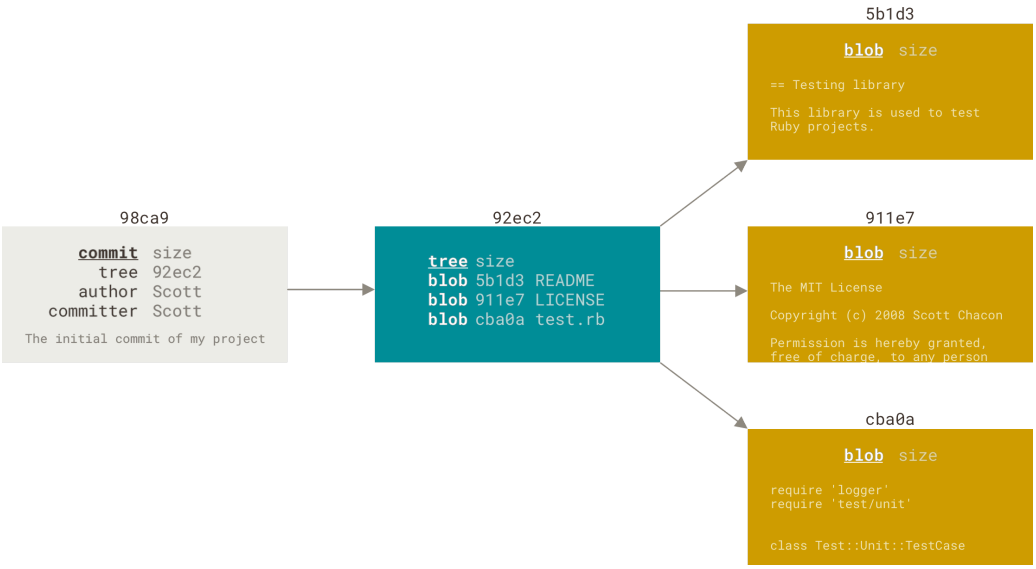


图 3　一个 Commit 通过 Tree 找到项目里的全部文件。来源：Pro Git。

好 Commit 应当小而完整，并且能让人读懂。提交说明可以参照这个示例：

```
fix(api): handle 401 response
```

冒号前写改动的类型与范围，冒号后用一句话写清这次改了什么。

2.2　Git 的区域划分

Git 把文件的变化分成三个区域处理。工作区存放正在编辑的文件。**暂存区**存放准备进入下一次 Commit 的变化。本地仓库保存已经提交的历史，位于项目目录下的 `.git` 文件夹。

图中从左到右是工作区，暂存区和本地仓库。从本地仓库指向工作区的箭头表示取出项目文件。从工作区指向暂存区的箭头表示选择变化。从暂存区指向本地仓库的箭头表示提交。

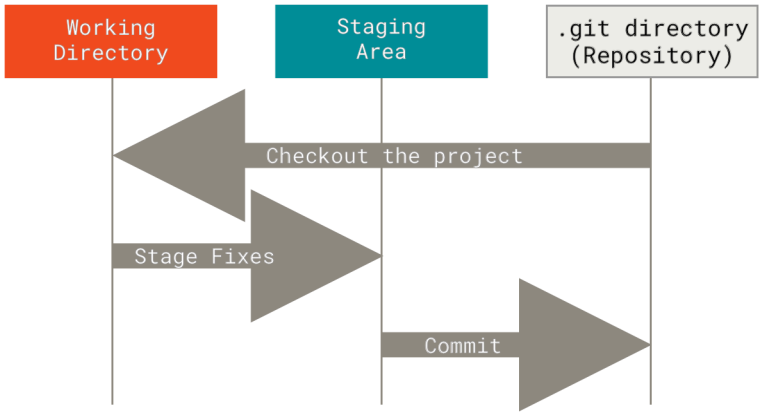


图 4　一次提交经过的三个区域。来源：Pro Git。

完整的提交路径是：文件先在工作区产生变化，再用 `git add` 选入暂存区，最后用 `git commit` 写入本地仓库。暂存区让开发者只挑选一部分变化放进下一次 Commit，每个 Commit 因而能保持小而完整。

2.3 分支的概念

分支是指向某个 Commit 的可移动标签。分支不是项目的副本，只是一个轻量的指针。在当前分支上产生新 Commit 时，分支随之向前移动，指向新的 Commit。

每个任务使用独立的分支，便于多人并行开发，也便于安全放弃：任务不再需要时，丢弃这个分支，其他分支不受影响。

图中下方的青色方框是快照，中间一排是 Commit，每个 Commit 向下指向自己的快照，向左指向父 Commit。右端的 Commit 最新，上方有两个指向这个 Commit 的红色标签，其中一个是分支。黄色的 HEAD 指向这个分支，表示当前所在的分支。

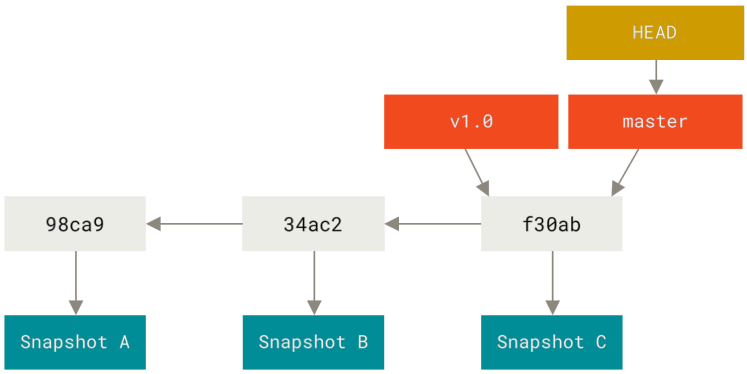


图 5　HEAD 指向当前分支，分支指向最新的 Commit。来源：Pro Git。

2.4 身份配置与获取仓库

第一次使用 Git 时，先配置作者身份。作者姓名与邮箱会写入之后的每个 Commit。

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

git config --global init.defaultBranch main
git config --global --list
```

前两行设置作者姓名与邮箱，引号里的内容换成自己的信息。第三行把新建仓库的默认分支名设为 `main`。最后一行列出全部全局配置，用来确认设置已经生效。

`--global` 表示全局配置，对这台电脑上的所有项目生效。公司账号与个人账号需要区分时，可以在项目目录里执行不带 `--global` 的同样命令，配置只对这个项目生效。提交邮箱应与 GitHub 账号关联，GitHub 才能正确显示贡献者身份。连接 GitHub 时使用 SSH Key 或受控的 Token 认证，密码与 Token 只保存在受控环境中。

获取仓库有两种方式：`git init` 创建一段新的历史，`git clone` 复制一段已有的历史。新项目用 `git init`：

```
mkdir customer-service
cd customer-service
git init
git status
```

`mkdir` 新建项目目录 `customer-service`。`cd` 进入这个目录。`git init` 在目录中创建新的 Git 仓库。`git status` 查看仓库当前的状态。

已有项目用 `git clone`：

```
git clone git@github.com:org/app.git
cd app
git remote -v
```

`git clone` 把 GitHub 上的仓库连同完整历史复制到本地，生成目录 `app`。`cd app` 进入这个目录。`git remote -v` 列出仓库配置的远程地址。团队开发默认使用 clone，因为一次就能同时得到完整历史与远程仓库配置。

2.5 最小提交循环

最小的提交循环从修改文件开始，先选择要提交的变化，再检查选中的内容，最后写入 Commit。

```
git status
git add docs/git-demo-notes.md
git diff --cached
git commit -m "docs: add Git workflow demo notes"
```

第一行的 `git status` 查看哪些文件发生了变化。第二行的 `git add` 把指定文件的变化选入暂存区。第三行的 `git diff` 加上 `--cached`，显示暂存区里准备提交的内容，用于提交前检查。第四行的 `git commit` 把暂存区的变化写入本地仓库，`-m` 后面引号里的文字是提交说明。

在这个循环中，文件的状态随之变化。新建的文件处于未跟踪状态，用 `git add` 加入后直接变为已暂存。已经提交过的文件处于未修改状态，编辑后变为已修改，选入暂存区后变为已暂存，提交之后回到未修改。图中左起第一列是未跟踪，第二列与第三列依次是未修改与已修改，第四列是已暂存，箭头标出状态之间的转换。

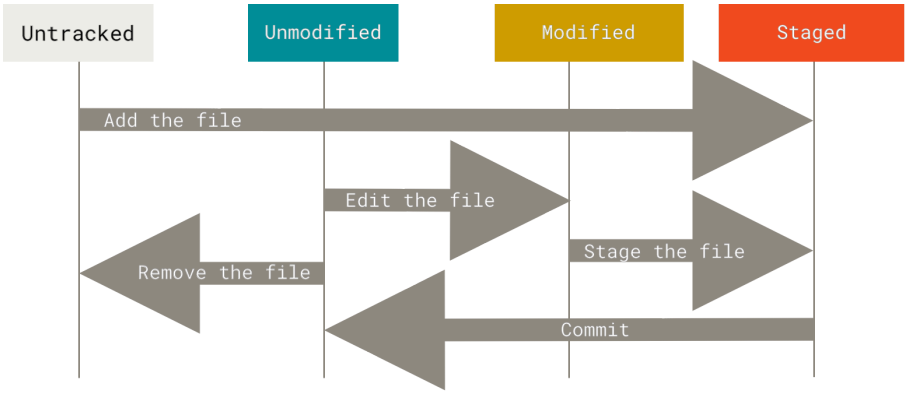


图 6　一个文件从未跟踪到提交所经历的状态变化。来源：Pro Git。

提交前后都要检查命令的输出，确认 Git 真正记录了什么。以 $ 开头的行是输入的命令，其余的行是输出。

```
$ git diff --cached --stat
 docs/git-demo-notes.md | 13 ++++++++++++
 1 file changed, 13 insertions(+)

$ git commit -m "docs: add Git workflow demo notes"
[main 0fd229a] docs: add Git workflow demo notes
 1 file changed, 13 insertions(+)
 create mode 100644 docs/git-demo-notes.md
```

`git diff --cached --stat` 只显示统计：暂存区里只有这一个文件，新增 13 行。`git commit` 输出的第一行 `[main 0fd229a]` 表示提交写在 `main` 分支上，新 Commit 的 ID 是 `0fd229a`。后两行说明这次提交新建了一个文件，共新增 13 行。

这时 Commit 只存在于本地仓库，GitHub 上的仓库还停留在上一个 Commit。

2.6 Commit 与 Push

Commit 与 Push 是两个独立的步骤。Commit 只记录本地历史，Push 才把这段历史同步到 GitHub。`origin` 是远程仓库的名字，这里指 GitHub 上的仓库，`origin/main` 表示远程 `main` 分支所在的位置。

Push 之前：

```
$ git status -sb
## main...origin/main [ahead 1]

$ git log --oneline --decorate -2
0fd229a (HEAD -> main) docs: add Git workflow demo notes
8ce893a (origin/main) feat: add deployment and operations lessons
```

`git status -sb` 用简短的格式显示状态，并带上分支信息。输出第一行末尾的 `[ahead 1]` 表示本地 `main` 比 `origin/main` 多一个 Commit。`git log` 的 `--oneline` 让每条提交只占一行，`--decorate` 在括号里标出分支所在的位置，`-2` 表示只显示最近两条提交。从输出看，本地 `main` 指向 `0fd229a`，`origin/main` 仍停在 `8ce893a`。

Push 之后：

```
$ git push
 8ce893a..0fd229a  main -> main

$ git status -sb
## main...origin/main
```

`git push` 把本地 `main` 推送到远程 `main`，输出中的 `8ce893a..0fd229a` 表示远程 `main` 从 `8ce893a` 前进到 `0fd229a`。再次查看状态时，`[ahead 1]` 已经消失，本地与 GitHub 保持一致。

2.7 阅读提交历史

阅读历史时，先查看提交的顺序，再查看某一次提交的具体变化。

```
$ git log --oneline --decorate --graph -4
* 0fd229a (HEAD -> main, origin/main) docs: add Git workflow demo notes
* 8ce893a feat: add deployment and operations lessons
* bef37be feat: add Git collaboration lessons
* ab2afa2 chore: initialize Day 5 beamer project

$ git show --stat 0fd229a
 docs/git-demo-notes.md | 13 ++++++++++++
 1 file changed, 13 insertions(+)
```

`--graph` 在每行前面用 `*` 画出提交的先后关系，`-4` 表示只显示最近四个 Commit，新的在上，旧的在下。第一行括号里同时出现 `HEAD -> main` 与 `origin/main`，表示本地 `main` 与 GitHub 上的 `main` 指向同一个 Commit，两者已经对齐。`git show --stat` 列出 `0fd229a` 这次提交改动了哪些文件。完整输出在文件统计之前还有提交说明等信息，示例只保留了文件统计。

查看具体改动还可以用两条命令：

```
git show 0fd229a
git diff ab2afa2..0fd229a
```

两条命令中，前一条显示 `0fd229a` 这次提交的说明与完整 Diff，Diff 逐行列出增加和删除的内容。后一条比较 `ab2afa2` 与 `0fd229a` 两个历史节点，显示两者之间的全部变化。

2.8 分支的日常命令

任务分支从最新的 `main` 创建，合并后及时删除。

```
git switch main
git pull --ff-only

git switch -c feature/login-api
# edit, test, commit...

git switch main
git branch -d feature/login-api
```

`git switch main` 切换到 `main` 分支。`git pull --ff-only` 把 GitHub 上最新的提交同步到本地 `main`。`--ff-only` 只允许本地 `main` 直接前移到远程的位置，如果本地 `main` 上有远程没有的提交，命令会报错停止，不会自动合并。

`git switch -c feature/login-api` 创建名为 `feature/login-api` 的分支，并切换到这个分支。以 `#` 开头的行是注释，代表在任务分支上修改，测试与提交。最后两行在任务分支合并进 `main` 之后执行：先切换回 `main`，再用 `git branch -d` 删除这个已经合并的分支。

创建分支不会复制文件。图中两个红色的分支标签指向同一个 Commit。之后在新分支上提交，只有新分支向前移动。

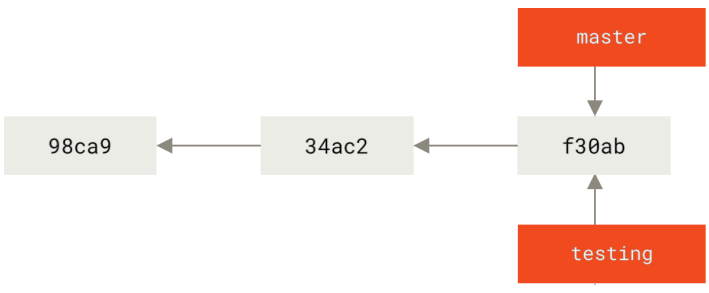


图 7　新建分支只是多了一个指向当前 Commit 的标签。来源：Pro Git。

2.9 撤销操作的选择

撤销之前，先判断要撤销的内容是否已经共享。已经推送的公开历史优先用 revert 撤销，还没有共享的历史才考虑重写。

场景	推荐命令	影响
未暂存的修改有误	`git restore file`	丢弃本地变化
误加入暂存区	`git restore --staged file`	保留文件变化
最近 Commit 的信息有误	`git commit --amend`	重写未共享提交
已推送的 Commit 有误	`git revert <commit>`	新增反向提交，历史安全

表中的 `file` 换成实际的文件路径，`<commit>` 换成要撤销的 Commit ID。四条命令的作用分别是：

- `git restore file` 丢弃工作区里还没有暂存的修改，文件回到最近一次暂存或提交时的内容，丢弃的修改无法找回。
- `git restore --staged file` 只把文件从暂存区撤回，文件里的修改仍然保留。
- `git commit --amend` 修改最近一次 Commit 的信息，这会重写这次提交，所以只用于还没有共享的提交。
- `git revert <commit>` 新增一个反向提交来抵消指定 Commit 的改动，原有历史保持不变。

共享分支上禁止两项操作：`reset --hard` 与普通的 `push --force`。两者都会重写或覆盖历史，普通的 `push --force` 还可能覆盖同事刚推送的 Commit。

小结

Git 的历史由一个个 Commit 组成，每个 Commit 保存完整快照并指向父 Commit。一次修改要经过选择，检查与提交才进入本地历史，推送之后才与团队共享。分支只是指向 Commit 的轻量标签，每个任务都可以使用独立分支。撤销的方式取决于历史是否已经共享，共享过的历史优先用新的反向提交修正。

03
GitHub 小团队协作流程

小团队协作以任务分支与小步提交为基础，开发期间持续同步 `main`，合并前由同伴检查。

3.1 小团队开发的完整流程

小团队开发的原则是每个任务对应一个分支，每个分支只有一个合并目标。完整流程分五步：

1. 明确任务边界与负责人。
2. 从 `main` 创建短生命周期分支。
3. 小步 Commit，并 Push 共享。
4. 同伴快速检查后合并。
5. 删除已完成的功能分支。

分支的生命周期短，与 `main` 的差距就小，检查与合并也更容易。

图中红色方块是分支名，灰色圆角框是 Commit。主分支之外有几条任务分支，每条从某个 Commit 分出后单独向前提交，互不干扰。

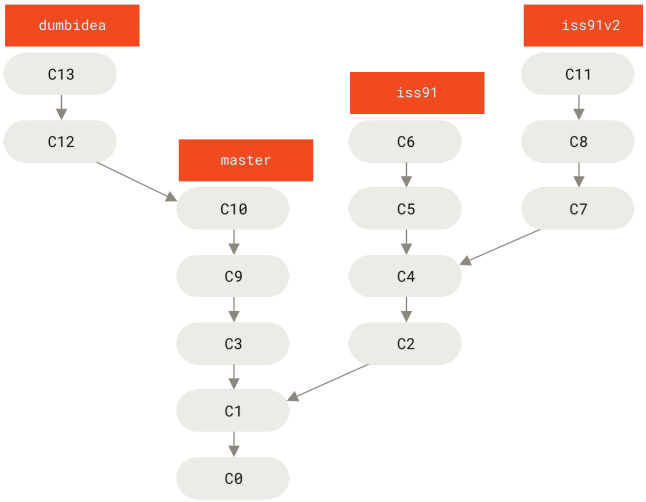


图 8　一个分支只承载一个清晰任务。来源：Pro Git。

3.2 开发者的日常命令流

开发者每天先同步 `main`，再创建任务分支，修改完成后提交并推送。

```
git switch main
git pull --ff-only
git switch -c feature/chat-history

# edit and test
git add src/
git diff --cached
git commit -m "feat(chat): load conversation history"

git push -u origin feature/chat-history
```

前三行切换到 `main`，同步 GitHub 上的最新提交，再创建任务分支并切换过去。注释行代表修改与测试。`git add src/` 把 `src/` 下的变化全部选入暂存区。`git diff --cached` 检查准备提交的内容。`git commit -m` 写入提交说明并完成提交。

最后一行把任务分支推送到 GitHub，`origin` 是远程仓库的名字，后面是要推送的分支名。`-u` 记录本地分支与远程分支的对应关系，之后在这个分支上直接执行 `git push` 即可推送。

推送之后，在 GitHub 上发起**合并请求**，由一名同伴快速检查后合并。

3.3 合并前快速检查

合并前的检查只围绕本次任务，重点是逻辑与测试，以及配置与密钥。检查清单共五项：

- 改动只包含当前任务。
- 核心逻辑与错误处理正确。
- 构建，测试或最小验证通过。
- 不包含 `.env`，密码和密钥。
- 接口，配置和数据库迁移的变化已在合并请求里说明。

检查通过并合并之后，删除这个功能分支。

检查的范围要受控：一次合并只解决一个问题，检查的工作量小，出了问题回滚的成本也低。

3.4 开发期间同步 main

功能分支开发期间，别人的改动会陆续合并进 `main`。功能分支开发超过半天时，应主动把最新的 `main` 同步到自己的分支。小团队优先使用 Merge 同步，全队采用统一的规则。

```
git fetch origin
git switch feature/chat-history
git merge origin/main

# resolve conflicts, test, commit
git push
```

第一行从 GitHub 取回最新的提交并更新 `origin/main`，本地分支不受影响。第二行切换到自己的功能分支。第三行把最新的 `main` 合并进功能分支。注释行表示出现冲突时先解决冲突，测试通过后再提交。推送任务分支时已经用 `-u` 记录了对应关系，所以最后直接执行 `git push`。

冲突在自己的分支上解决并测试，问题就不会留到最终合并时才暴露。

3.5 密钥的存放与泄漏处理

密钥必须留在 Git 之外。严禁提交的内容包括：

- `.env` 文件
- 数据库密码
- JWT Secret

- 云厂商密钥
- 模型 API Key
- 私钥文件

部署时，密钥通过服务器环境变量，Secret 文件或平台的 Secret 功能注入，不经过仓库。

提交前先用 `git diff --cached` 查看准备提交的内容，再用 Secret Scanner 扫描是否混入密钥。平时使用的密钥按最小权限授予，只拥有完成任务所需的权限，即使泄漏，影响范围也有限。

发现泄漏后，按顺序处理：

1. 立即吊销或轮换密钥。
2. 处理仓库文件，清理 Git 历史与 Fork。
3. 清理缓存和 CI 日志。
4. 检查泄漏原因。

顺序不能颠倒。删除文件并不能让密钥失效，Git 历史与 Fork 里仍保留着这个密钥，缓存和 CI 日志里也可能留有副本，所以先让泄漏的密钥作废，再做清理。

小结

小团队协作把每项任务放进独立的短期分支，经同伴检查后才合并回 `main`。合并前只检查本次任务的改动，范围越小，检查与回滚越容易。开发时间较长时，定期把最新的 `main` 合并进来，冲突留在自己的分支里解决。密钥不进入仓库，一旦泄漏，先作废再清理。

04
冲突处理与 Review 及主分支保护

多人修改同一个项目时可能产生冲突。处理冲突要读懂冲突标记，按步骤解决，并用保护规则约束 `main`。

4.1 冲突标记的读法

两个分支修改了同一处代码，合并时 Git 无法自动决定保留哪一侧，就会产生冲突，并在文件里写入**冲突标记**。冲突标记把冲突的位置分成三部分：当前分支的内容，分隔线和待合并分支的内容。

```
<<<<<<< HEAD
const apiBase = "/api";
=======
const apiBase = "http://localhost:8000";
>>>>>>> feature/api-config
```

`<<<<<<< HEAD` 与 `=======` 之间是当前分支的内容。`=======` 是分隔线。分隔线与 `>>>>>>>` 之间是待合并分支的内容，`>>>>>>>` 后面写着这个分支的名字。例子里两侧给 `apiBase` 取了不同的值：当前分支用 `/api`，待合并分支用本机的 8000 端口地址。

选择哪一侧，或者怎样融合两侧，取决于生产架构与配置约定，需要先确认清楚再修改。演示项目约定 `/` 访问前端，`/api` 访问后端。删除全部冲突标记后，必须重新运行测试与构建。

4.2 命令行解决冲突的标准步骤

命令行解决冲突按固定的四步进行。

```
git status
# open conflicted files and resolve markers

pnpm lint
pnpm test
pnpm build

git add src/config/api.ts
git commit                 # merge workflow
# or: git rebase --continue
```

1. 用 `git status` 定位冲突，命令会列出所有未合并的路径，即发生冲突的文件。
2. 打开这些文件，逐处编辑冲突标记。
3. 依次运行 `pnpm lint`，`pnpm test` 和 `pnpm build`，验证 Lint 检查，测试与构建都能通过。
4. 用 `git add src/config/api.ts` 把处理好的文件加入暂存区，表示这个文件的冲突已经解决，然后继续：Merge 流程执行 `git commit` 完成合并，Rebase 流程执行 `git rebase --continue`。

解决冲突时必须理解两侧改动的业务意图。不确定怎样处理时，可以放弃这次操作，回到操作之前的状态，Merge 流程用第一条命令，Rebase 流程用第二条命令：

```
git merge --abort
git rebase --abort
```

4.3 GitHub 在线解决简单行冲突

两个分支都改动过的同一行称为**竞争行**。简单的竞争行冲突可以直接在 GitHub 网页的冲突编辑器里解决。图中是这个编辑器的顶部：左边是发生冲突的文件路径，中间显示这个文件的冲突数量与跳到上一个，下一个冲突的按钮，右边橙色框标出的按钮用来把冲突标记为已解决。



图 9　GitHub 网页冲突编辑器的顶部工具栏。来源：GitHub Docs。

复杂冲突建议回到本地处理，例如删除与重命名，生成文件和依赖锁文件。演示项目的依赖锁文件就是 `pnpm-lock.yaml` 与 `uv.lock`。无论在网页上还是在本地解决，合并前都必须验证构建，测试和关键业务流程。

4.4 减少冲突的做法

减少冲突的重点在工作拆分，同步和接口约定，具体做法有五条：

- 把工作拆成小任务，使用短生命周期分支。
- 明确模块负责人，减少多人同时修改核心文件。
- 频繁同步 `main`，让竞争的变化尽早暴露。
- Schema，路由和公共配置先约定再编码。
- 格式化与依赖更新单独提交。

前两条属于工作拆分，第三条属于同步，第四条属于接口约定。最后一条让格式化与依赖更新不和功能改动混在同一个 Commit 里，每个 Commit 只做一件事，检查与回滚都更清楚。

4.5 Rebase 与历史重写

Merge 与 Rebase 都能把 `main` 上的新变化同步到功能分支，两者对历史的处理不同。Merge 保留分叉的历史。Rebase 把功能分支上的 Commit 复制到新的父节点上，生成新的 Commit。

图中红色方块是分支名。任务分支与主分支从同一个 Commit 分叉，各自前进后用 Merge 合并，合并产生的新 Commit 同时连接两条分支的最新 Commit。

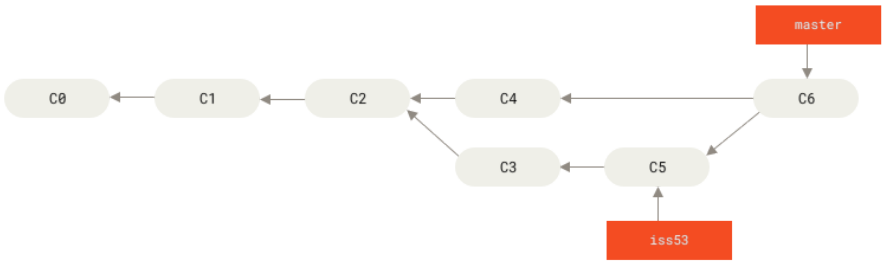


图 10　Merge 产生的合并提交有两个父 Commit。来源：Pro Git。

Rebase 复制出的是新的 Commit，Commit ID 随之改变。如果这些 Commit 已经推送过，再推送就需要重写远程历史。所以 Rebase 只用来重写自己的功能分支，共享的历史保持稳定。

Rebase 之后推送，使用 `--force-with-lease`：

```
git fetch origin
git rebase origin/main

git push --force-with-lease
```

第一行取回最新的 `origin/main`。第二行把当前功能分支的 Commit 复制到最新的 `origin/main` 之上。最后一行推送重写后的历史，推送前先检查远程分支是否被别人更新过，如果有人更新过，就拒绝推送。

`git push --force` 不做这项检查，可能覆盖同事刚推送的 Commit，因此禁止使用。如果团队还没有理解历史重写，统一用 Merge 同步 `main` 更安全。

4.6　主分支保护规则

`main` 始终代表“可部署版本”。为此，GitHub 上的主分支要配置保护规则：

- 禁止直接 Push，所有变化通过受控的合并流程进入 `main`。
- 至少一人 Approve，关键目录可以配置 CODEOWNERS。
- 必须通过构建，测试，Lint 等状态检查。
- 必须解决 Review 对话后才能合并。
- 默认禁止 Force Push 与删除主分支。

Approve 指同伴在合并请求上给出批准。CODEOWNERS 为关键目录指定负责人。状态检查是平台在合并前自动运行的检查，结果全部通过才允许合并。Review 对话是检查者在合并请求里留下的讨论，每一条都要解决。

这些规则把“团队口头约定”变成平台自动执行的门禁，不满足条件的合并会被平台阻止。规则内容参考 GitHub Docs 的 About protected branches。

小结

冲突出现时，先读懂标记两侧分别来自哪个分支，再结合业务意图决定取舍，改完必须重新测试与构建。简单的行冲突可以在网页上处理，涉及文件结构或锁文件的冲突应回到本地。Rebase 会生成新的 Commit，只适合自己的功能分支，推送时以 `--force-with-lease` 代替 `--force`。主分支保护规则让每一次进入 `main` 的改动都经过审批与自动检查。

05
服务器购买与部署架构

公网部署从一台持续运行的服务器开始。服务器买好后，先按端口开放策略配置云防火墙，再在宝塔与 Docker 两条部署路线中做出选择。

5.1 购买服务器

购买云服务器时，要在四个方面做出选择：

选项	建议
地域	靠近用户，境内上线关注备案
系统	Ubuntu 或 Debian 的 LTS 纯净镜像
配置	练习项目建议从 2C4G 起步
网络	公网 IPv4 与带宽，开放 22，80 与 443 端口

地域选在靠近用户的位置。系统使用 Ubuntu 或 Debian 的 LTS 纯净镜像，纯净镜像不预装多余的软件，运行环境由开发者按项目需要自行安装。

服务器买好后，记录四项信息：

- 公网 IPv4 地址，SSH 用户与密钥。
- 实例名称与到期时间。
- 云防火墙规则。
- 控制台的重置与救援入口。

这些信息各有用途。登录服务器要用公网 IPv4 地址与 SSH 密钥。续费要看实例的到期时间。开放端口要修改云防火墙规则。服务器无法登录时，从控制台的重置与救援入口处理。

服务在境内上线前，应依法判断是否需要备案或许可。相关规定见工信部的“非经营性互联网信息服务备案管理办法”。

5.2 端口开放策略

服务器在公网上只保留 22，80 与 443 三个端口。22 端口用于 SSH 管理，80 端口用于证书验证与跳转 HTTPS，443 端口是用户正式访问的入口。

应用自己的端口不向公网开放。Node 与 Next.js 常用 3000 端口，FastAPI 常用 8000 端口，这两个端口默认只在本机或容器网络里访问。MySQL 的 3306 端口同样不向公网开放，应用通过本机或内网连接数据库。所有公网请求先到达 Nginx，再由 Nginx 转发给本机的应用。

端口	典型用途	建议
22	SSH	仅管理使用，可限制来源 IP，使用密钥登录
80	HTTP	证书验证与跳转 HTTPS
443	HTTPS	用户正式访问入口
3000	Node 与 Next.js	默认仅本机或容器网络
8000	FastAPI	默认仅本机或容器网络
3306	MySQL	不向公网开放，应用走本机或内网

端口规则在云防火墙里配置，具体操作可参考阿里云轻量应用服务器的防火墙文档。

5.3 宝塔与 Docker 的分工

部署应用有两条路线。**宝塔部署**通过可视化面板管理服务器，容易上手。**Docker 部署**用镜像描述运行环境，部署过程可以重复。两条路线的差别：

维度	宝塔部署	Docker 部署
学习门槛	面板可视化，容易上手	需要理解镜像，网络与卷
环境一致性	依赖服务器当前状态	镜像描述运行环境
迁移复现	需要记录手工步骤	Dockerfile 与 Compose 可重复执行
排错入口	面板日志与系统日志	`docker logs` 与容器状态
适用场景	新手与小型单机项目	团队交付，多服务与持续迭代

宝塔部署的结果取决于服务器当前的状态，换一台服务器时，要照着记录下来的手工步骤重做一遍。Docker 部署把运行环境写进镜像，在新服务器上执行同样的 Dockerfile 与 Compose 文件，就能得到相同的运行环境。

两条路线也可以组合使用：宝塔管理 Nginx 与证书，Docker Compose 运行应用。

小结

选购服务器时，地域靠近用户，系统选用 LTS 纯净镜像。购买后，把登录凭据，到期时间与救援入口记录在案。公网只开放 SSH 管理与网站访问所需的三个端口，应用和数据库的端口留在内部，由 Nginx 统一对外。宝塔部署依赖面板与手工步骤，适合入门和单机项目。Docker 部署把环境写进镜像，适合团队交付与多服务项目。两者可以分工配合，宝塔负责入口与证书，Compose 负责运行应用。

06
宝塔部署的关键知识点

宝塔面板把服务器上的常见操作做成了可视化界面，面板上的每个选项，背后都对应着站点配置，进程管理与反向代理规则。理解这些对应关系，才能在面板里做出正确的选择，出问题时也能判断故障出在哪一层。

6.1 环境组件与职责

前后端分离的项目部署到宝塔，要先装好几类环境组件。每个组件负责一层工作，先理解分工，再安装。

组件	用途	选择建议
Nginx	80 与 443 端口，静态文件，反向代理	必装
Node.js	构建前端，运行 Node 后端或 SSR	安装项目要求的 LTS 版本
pnpm	Node 依赖与构建	通过 Corepack 或官方方式启用
Python 与 uv	FastAPI 运行环境	与 `pyproject.toml` 要求一致
MySQL	业务数据	本机部署时仅监听本地或内网

Nginx 是必装组件，负责接收 80 与 443 端口上的请求，直接返回前端的静态文件，并把接口请求转发给后端。Nginx 代替后端接收请求，再转发给后端，这种方式称为**反向代理**。Node.js 负责构建前端，也负责运行 Node 后端或 SSR 项目。pnpm 管理 Node 依赖并执行构建。Python 与 uv 提供 FastAPI 的运行环境。MySQL 保存业务数据，本机部署时只监听本地或内网地址。

各组件的版本以仓库里的声明为准：Python 环境与 `pyproject.toml` 的要求一致，Node.js 安装项目要求的 LTS 版本。生产环境的升级要事先计划。

6.2 静态前端与 SPA 路由

用 Vite 构建的 React 或 Vue 项目，构建后生成一组静态文件，称为**构建产物**，生产构建通常输出到 `dist/` 目录。在宝塔里部署静态前端，网站根目录要指向 `dist/` 这样的构建产物目录，不能指向 `src/` 或项目根目录。Nginx 直接把构建产物返回给浏览器，不运行前端源码，服务器上也不需要常驻的 Node 进程。

前端代码请求后端时使用相对路径 `/api`。请求发往页面所在的同一个域名，再由这个域名下的 Nginx 转发给后端。

构建时注入的环境变量会写进静态文件，用户在浏览器里就能看到，所以其中不能保存 Secret。环境变量或源码变化后，要重新构建，并用新的构建产物替换旧的。

部署前先判断项目类型。静态构建的产物由 Nginx 直接返回，Next.js 与 Node SSR 项目则要由 Node.js 运行，两类项目的部署方式不同。

SPA 指单页应用。React Router 与 Vue Router 的 History 模式由浏览器管理路由，地址栏里的 `/chat/123` 这类路径由前端代码解析，服务器上并没有同名文件。用户直接访问或刷新 `/chat/123` 时，请求发到 Nginx，Nginx 找不到同名文件，就返回 404。修复的办法是让未知的前端路由回到 `index.html`：

```
location / {
    try_files $uri $uri/ /index.html;
}
```

`try_files` 按顺序尝试三个目标：先找与请求路径同名的文件 `$uri`，再找同名目录 `$uri/`，都找不到时返回 `/index.html`。`index.html` 加载后，前端路由接管，按地址显示对应的页面。`/api` 的请求不能落进这条回退规则，要保持独立的代理规则。

6.3 FastAPI 运行模型与进程守护

FastAPI 后端在宝塔里的运行模型可以概括为三点：锁定依赖，仅本机监听，交给进程守护。

关键点	生产环境含义
依赖	遵循 `pyproject.toml` 与 `uv.lock`
启动入口	使用项目真实的 ASGI 入口
监听地址	仅监听 `127.0.0.1:8000`
健康检查	提供轻量 `/health`
生命周期	开机启动，异常重启与日志

依赖按 `pyproject.toml` 与 `uv.lock` 安装，服务器上的依赖版本与开发环境一致。启动时使用项目真实的 ASGI 入口，例如 `app.main:app`。后端只监听 `127.0.0.1:8000`，公网无法直接访问，请求要经过 Nginx 转发。`/health` 是一个轻量的健康检查接口，用来确认后端进程是否正常。生产环境里，一个服务可能由多个独立进程共同承担请求。

部署分两个阶段：先证明应用能启动，再交给守护进程长期运行。**进程守护**让应用在 SSH 断开后继续运行，异常退出后自动恢复。配置守护时要填写六类内容：

- 工作目录与真实启动入口。
- Python 虚拟环境或 Node 版本。
- 本机监听端口与运行用户。
- 环境变量和受控的 Env 文件。
- 开机启动与异常退出重启。
- 标准输出与错误日志，以及日志的保留周期。

守护工具关注的信息包括进程的运行用户，资源占用和退出状态。不同版本的宝塔，守护功能的入口名称不同，可能显示为 Python 项目，应用管理器或进程守护。

6.4 Nginx 反向代理与面板字段

前端与后端部署在同一个域名下，由 Nginx 把 `/api` 开头的请求转给 FastAPI。同域部署让浏览器把接口请求当作同源请求，CORS 与 Cookie 方面的问题随之减少。

```
location /api/ {
    proxy_pass http://127.0.0.1:8000/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

`location /api/` 匹配以 `/api/` 开头的请求。`proxy_pass` 把匹配到的请求转给本机 8000 端口上的 FastAPI，例如访问 `https://app.example.com/api/health` 时，请求由 Nginx 转给 FastAPI。四行 `proxy_set_header` 把外部请求的信息交给后端：`Host` 传递域名，`X-Real-IP` 与 `X-Forwarded-For` 传递客户端 IP，`X-Forwarded-Proto` 传递外部使用的协议。后端启用 Proxy Headers 后，才能正确识别外部协议与客户端信息。

`proxy_pass` 地址末尾的斜杠会影响路径改写。地址末尾带斜杠时，Nginx 去掉匹配到的 `/api/` 前缀再转发，`/api/health` 到了后端变成 `/health`。地址末尾不带斜杠时，路径原样转发。两种写法下后端收到的路径不同，配置后必须现场验证真实路由。

宝塔的反向代理界面是 Nginx 配置的可视化写法，面板里填写的字段最终都会写成 Nginx 配置：

面板字段	对应配置	作用
代理目录	`location`	决定哪些请求被匹配
目标 URL	`proxy_pass`	指向本机应用端口，例如 `http://127.0.0.1:3000`
Host 与转发头	`proxy_set_header`	保留域名，协议和客户端信息

接口请求通常关闭缓存，静态资源可以单独配置缓存。WebSocket 与 SSE 还需要额外确认连接与超时设置。

6.5 前后端联调的验证顺序

前后端联调按后端，Nginx，浏览器的顺序逐层检查。前两步在服务器上执行：

```
curl http://127.0.0.1:8000/health
curl -i http://127.0.0.1/api/health
```

第一条命令绕过 Nginx，直接请求本机 8000 端口上的健康检查接口，确认后端进程是否正常。第二条命令请求本机 80 端口上的 `/api/health`，请求先经过 Nginx 再转给后端，用来确认 Nginx 反向代理是否正常。`-i` 让输出带上响应头。

完整的验证顺序有五步：

1. 用第一条命令确认后端进程正常。
2. 用第二条命令确认 Nginx 反向代理正常。
3. 浏览器打开域名，确认前端静态资源正常。
4. 按 F12 打开 Network 面板，查看 `/api/...` 请求的状态码与响应体。
5. 查看后端日志，确认同一请求是否到达，以及耗时与报错情况。

定位的方法是：在哪一层第一次失败，问题就位于这一层或这一层的下一跳。例如第一步成功而第二步失败，问题出在 Nginx 的转发配置，或者 Nginx 到后端的这一跳。

6.6 Node 后端与项目配置

后端换成 Node 时，运行时不同，部署模型相同。

对比项	Node 后端	FastAPI 后端
依赖锁定	`pnpm-lock.yaml`	`uv.lock`
构建阶段	NestJS 或 TypeScript 项目常需构建	通常直接运行 Python 源码
运行时	Node.js，pnpm 与 PM2	Python 虚拟环境与 ASGI Server
本机端口	常见 `127.0.0.1:3000`	常见 `127.0.0.1:8000`
进程管理	宝塔 Node 项目或 PM2	Python 项目或应用管理器

两种后端的公网入口相同，都由 Nginx 反向代理，不直接暴露业务端口。两者共用一套部署模型，包含四个要素：

- 用锁文件锁定依赖。
- 应用只监听本机端口。
- 进程交给守护工具。
- 公网入口统一由 Nginx 提供。

在宝塔里添加 Node 项目，要认清项目路径与启动命令，以及端口与运行用户。添加窗口的 PM2 项目页先填项目名称与 Node 版本，再选择启动文件与运行目录。图中红色箭头指向启动文件一栏，这里选择项目对应的 PM2 配置文件，运行目录通常就是启动文件所在的目录。

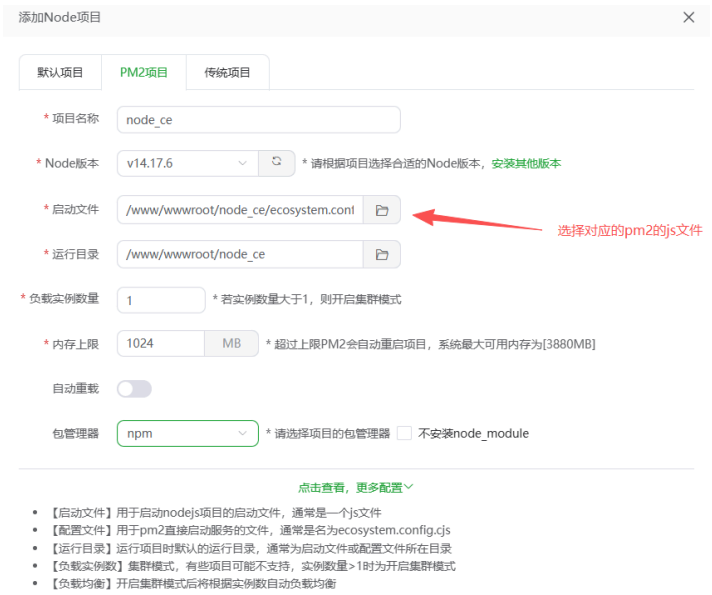


图 11　宝塔添加 Node 项目的窗口，填写 Node 版本，启动文件与运行目录。来源：宝塔官方文档。

同一窗口往下配置运行用户与项目端口。图中项目端口填 3000，旁边的放行端口没有勾选，业务端口不向公网开放。运行用户一栏的提示是无特殊需求时选择 www 用户。



图 12　同一窗口里配置运行用户与项目端口。来源：宝塔官方文档。

执行 pnpm 时提示找不到命令，先确认 Node 版本与 PATH 环境变量，避免重复安装。

6.7 服务器上的环境变量

环境变量的原则是仓库保存变量名，服务器保存真实值。不同类型的配置，处理方式不同：

类型	处理方式
前端公开配置	构建时注入，默认视为用户可见，不放 Secret
后端普通配置	项目环境变量或权限受控的 Env 文件
数据库密码与 API Key	最小权限，定期轮换，日志脱敏
JWT Secret	使用足够随机的值，变更会影响现有 Token
上传目录	独立持久目录，权限给运行用户并纳入备份

前端公开配置在构建时写进静态文件，访问网站的人都能看到，只能放可以公开的内容。数据库密码与 API Key 按最小权限分配，定期轮换，写入日志时要脱敏。JWT Secret 要用足够随机的值，修改后，已经签发的 Token 会受影响。上传目录放在独立的持久目录，权限交给运行用户，并纳入备份。

Env 文件只允许部署用户或运行用户读取，也不能放在静态站点能访问到的位置，避免通过静态站点暴露。

小结

宝塔部署把全栈应用分成几层：Nginx 对外接收请求并返回静态文件，后端在本机端口上运行，由守护工具保持长期运行。前端部署的是构建产物，单页应用还要用回退规则把未知路由交给 `index.html`。接口请求经 `location /api/` 转发，转发地址末尾有没有斜杠，决定后端收到的路径。联

调由内向外进行，从后端进程一直查到浏览器，第一次出错的位置就是排查起点。真实的密钥只存放在服务器上，并严格限制读取权限。

07
Docker 基础与部署模型

Docker 用镜像描述运行环境，用 Dockerfile 与 Compose 让部署过程可以重复。掌握 Docker 的核心概念，才能读懂 AI 生成的部署文件，也能自己排查容器问题。

7.1 Docker 核心部件的关系

Docker 的核心部件有六个：

部件	作用
Dockerfile	构建说明，描述镜像怎样构建
Image	镜像，只读的模板
Container	容器，镜像的运行实例
Registry	镜像仓库，存放可以推送与拉取的镜像
Volume	数据卷，持久化保存数据
Network	网络，负责服务之间的通信

部件之间按一条链路协作。`docker build` 按 Dockerfile 构建出**镜像**，镜像可以推送到 Registry，也可以从 Registry 拉取。`docker run` 用镜像启动**容器**，同一个镜像可以启动多个容器。容器把需要持久保存的数据放进**数据卷**，通过 Network 与其他容器通信。图中每条箭头上都标出了对应的动作，例如 `docker build` 与 `docker run`。

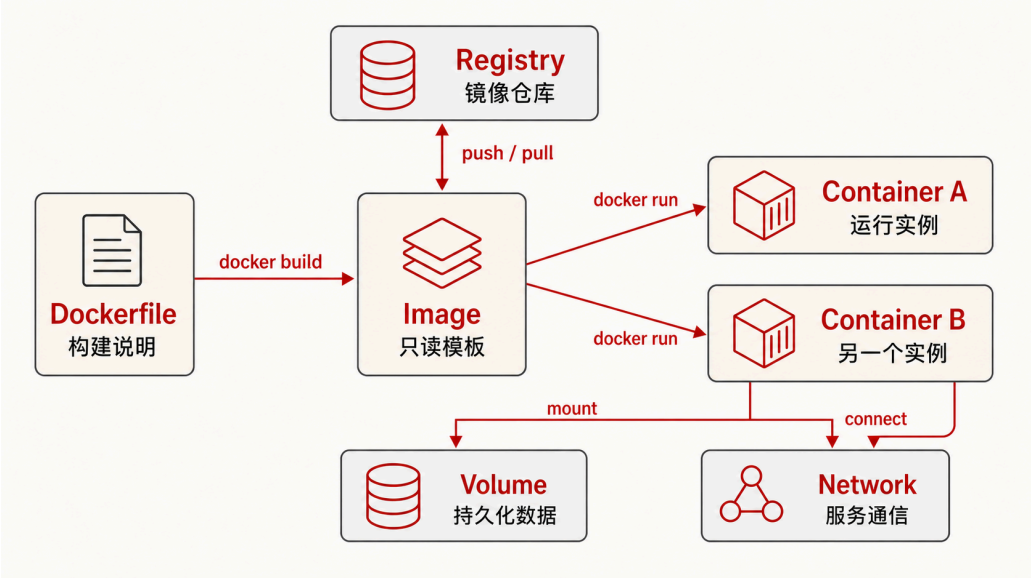


图 13　同一个镜像可以启动多个容器，Volume 保存数据，Network 负责容器通信。

7.2　最小 Dockerfile 读法

Dockerfile 的每条指令都在描述镜像的构建过程。一个最小的 Dockerfile 用七条指令就能为 FastAPI 应用构建镜像：

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "-m", "uvicorn", "app.main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

`FROM python:3.12-slim` 指定基础镜像。`WORKDIR /app` 把容器里的工作目录设为 `/app`。`COPY requirements.txt .` 先把依赖清单复制进去，`RUN pip install --no-cache-dir -r requirements.txt` 按清单安装依赖。`COPY . .` 再把项目源码复制进镜像。`EXPOSE 8000` 声明容器内服务使用的端口。`CMD` 写明容器启动时执行的命令，用 uvicorn 启动 `app.main:app`，监听 `0.0.0.0` 的 8000 端口。

`RUN` 在构建镜像时执行，`CMD` 在容器启动时执行。容器内的服务要监听 `0.0.0.0`，才能从容器网络访问。宝塔路线里后端只监听 `127.0.0.1`，放进容器后，监听地址要改为 `0.0.0.0`。`CMD` 写成 JSON 数组的形式，称为 JSON Exec Form，这种写法能更正确地接收停止信号。

示例为了便于阅读，用 `requirements.txt` 与 `pip` 安装依赖。演示项目的 FastAPI 后端用 uv 与 `uv.lock` 锁定依赖，生产环境的 Dockerfile 按锁文件安装。

7.3 Docker 层与缓存

镜像由一层层只读层叠成，每条构建指令形成一层。以 Python 应用的镜像为例，各层从上到下依次是：

层	内容
顶层	应用源码
第四层	安装好的依赖
第三层	应用的依赖清单
第二层	Python 与 pip
底层	Debian 基础镜像

越往上的层，内容变化越频繁。

再次构建时，没有变化的层可以直接复用，这就是构建缓存。某一层一旦发生变化，这一层和后面所有的层都要重新构建。因此 Dockerfile 要把变化少的步骤放前面，顺序是：

1. 复制依赖清单与锁文件。
2. 安装依赖，生成可缓存的层。
3. 再复制频繁变化的源码。
4. 最终镜像只保留运行产物。

最小 Dockerfile 也按这个顺序编写：先复制 `requirements.txt` 并安装依赖，再用 `COPY . .` 复制源码。常见的错误是先执行 `COPY . .` 再安装依赖，这样源码的任何改动都会让依赖层的缓存失效，每次构建都要重新安装全部依赖。最终镜像只保留运行产物，靠的是多阶段构建：前面的阶段负责安装与构建，最后一个阶段只复制运行需要的产物。

7.4 构建运行与端口映射

镜像构建好之后，用 `docker run` 启动容器，并把容器端口映射到宿主机。

```
docker build -t customer-api:1.0 .

docker run -d \
  --name customer-api \
  -p 127.0.0.1:8000:8000 \
  --restart unless-stopped \
  customer-api:1.0
```

`docker build -t customer-api:1.0 .` 用当前目录的 Dockerfile 构建镜像，`-t` 给镜像起名 `customer-api` 并打上 Tag `1.0`，末尾的 `.` 表示使用当前目录。生产镜像使用明确的 Tag。

`docker run` 用这个镜像启动容器。`-d` 让容器在后台运行。`--name customer-api` 给容器命名，排错命令用这个名字指定容器。`--restart unless-stopped` 让容器退出后自动重启，手动停止的容器除外。最后一行指定要运行的镜像 `customer-api:1.0`。

`-p 127.0.0.1:8000:8000` 负责端口映射：左侧 `127.0.0.1:8000` 是宿主机的地址与端口，右侧 `8000` 是容器监听的端口。宿主机一侧绑定 `127.0.0.1`，端口只对本机开放，公网访问仍要经过 Nginx。

这些命令由客户端交给宿主机上的 Docker 服务执行。图中左边是客户端发出的 `docker run`，`docker build` 与 `docker pull` 三条命令。中间是宿主机，Docker 服务在这里管理镜像与容器。右边的 Registry 存放可以拉取的镜像。实线，点线与虚线分别对应 `docker run`，`docker build` 与 `docker pull` 的执行路径。

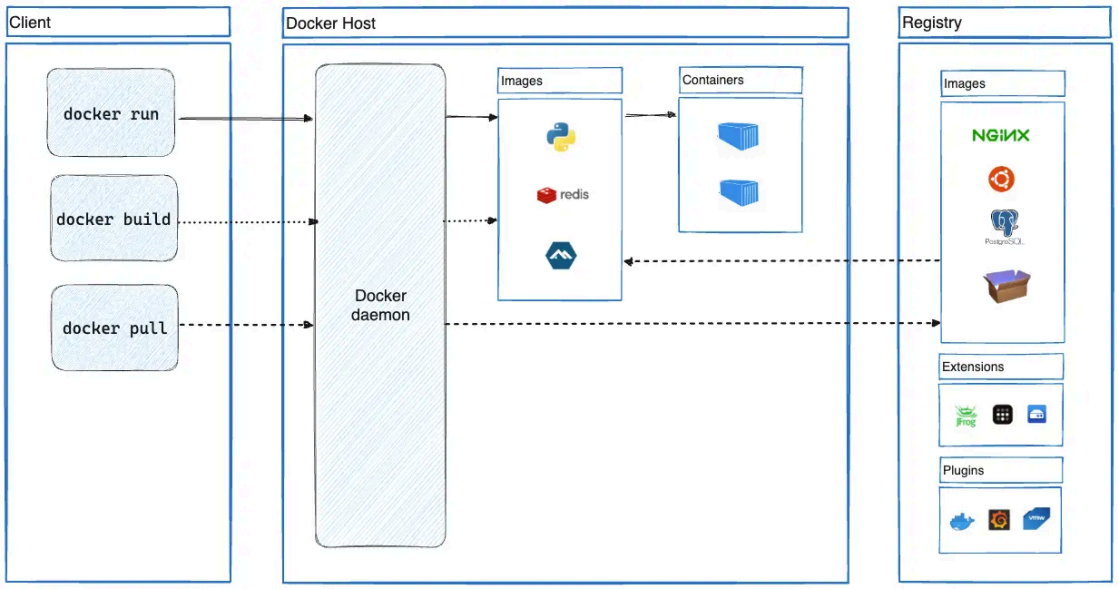


图 14　客户端把命令交给宿主机上的 Docker 服务，镜像可以从 Registry 拉取。来源：Docker Docs。

7.5　容器排错命令

容器出问题时，用这四条命令排查：

```
docker ps -a
docker logs --tail 200 -f customer-api
docker exec -it customer-api sh
docker inspect customer-api
```

`docker ps -a` 列出全部容器，包括已经退出的容器，用来看退出状态，判断容器是否陷入重启循环。`docker logs --tail 200 -f customer-api` 显示最近 200 行日志并持续跟踪新输出，日志内容是应用的标准输出与错误输出。`docker exec -it customer-api sh` 在运行中的容器里打开一个交互式 shell，用于在进程内检查。`docker inspect customer-api` 输出容器的完整配置，例如端口映射，环境变量与挂载的数据卷。

`docker exec` 只用于诊断，不应在容器里手工修改生产代码。容器重建后，在容器里做的手工修改会丢失，代码改动要通过重新构建镜像发布。

7.6 用服务名互访

Compose 会为项目创建默认网络，同一网络里的容器用服务名互相访问。演示项目有三个服务：frontend 运行 Nginx，通过 `http://api:8000` 访问后端。api 运行 FastAPI，监听 8000 端口，通过 `mysql://db:3306` 访问数据库。db 运行 MySQL，监听 3306 端口。

容器之间不使用 `localhost` 互访。`localhost` 只代表当前容器自己，在 api 容器里访问 `localhost:3306`，指向的是 api 容器自身，无法连接 db 容器里的 MySQL。服务名 `api` 与 `db` 就是网络里的主机名。

只有需要给宿主机或公网访问的服务才映射 ports，其余服务只在容器网络内部访问。

7.7 Compose 管理多容器应用

Docker Compose 用一份 YAML 文件描述多个服务，以及服务用到的网络，卷与配置。Compose 适合单机部署，也是全栈 Docker 部署路线的核心工具。常用命令有六条：

```
docker compose config
docker compose build
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down
```

`docker compose config` 先检查合并后的配置和变量替换结果，配置写错时在这一步就能发现。`docker compose build` 构建各服务的镜像。`docker compose up -d` 在后台启动全部服务。`docker compose ps` 查看各服务的状态。`docker compose logs -f` 持续跟踪各服务的日志。`docker compose down` 停止并删除项目的容器，默认不删除 Named Volume，数据卷里的数据会保留。`down` 加上 `-v` 会删除数据卷，数据库的数据也会一起删掉，执行前必须谨慎。

7.8 用 Claude Code 准备部署

理解了 Docker 的核心概念，就可以让 Claude Code 扫描仓库，为生产服务器准备整套 Docker 部署文件。提示词先交代角色，任务与已知技术栈，再按顺序列出八项工作，并写明约束。八项工作从检查项目现状开始，经过逐个创建部署文件，到给出运维命令结束。

其中四项要求直接对应 Docker 的基本做法：

- Dockerfile 采用多阶段构建，按锁文件安装依赖。
- `.dockerignore` 排除 `.git` 与 `.env`。
- MySQL 的 `/var/lib/mysql` 挂载到 named volume。

- 只把网关绑定到宿主机的 `127.0.0.1`。

约束部分为 AI 划定边界，例如不猜测缺失的业务信息，不提交 `.env` 与密钥，交付前执行 `docker compose config` 自检。完整的提示词：

```
你是资深DevOps 工程师。请先扫描当前仓库，再为生产服务器准备Docker 部署。

已知技术栈：
- 前端：Node.js + pnpm；判断是静态构建还是Next.js / Node SSR。
- 后端：Python FastAPI + uv，必须遵循pyproject.toml 与uv.lock。
- 数据库：MySQL 8.4；公网入口由Nginx 提供，同域/api 转发后端。
请按顺序完成：
1. 检查目录、构建脚本、启动命令、端口、环境变量、迁移、健康检查和上传目录。
2. 先输出部署拓扑、假设、风险与将要新增/修改的文件，然后直接实现。
3. 为每个服务创建生产级Dockerfile：多阶段构建、锁文件安装、最小运行镜像。
4. 创建.dockerignore，排除.git、.env、依赖、缓存、构建物和本地虚拟环境。
5. 创建compose.yaml：frontend、api、db（镜像固定mysql:8.4）、healthcheck、restart、内部networ
k；将MySQL /var/lib/mysql 挂载到named volume；只把网关绑定到宿主机127.0.0.1。
6. 创建Nginx 配置：SPA fallback 或SSR 反代、/api、代理头、WebSocket / SSE。
7. 创建.env.example，只保留变量名和安全示例，禁止写入任何真实Secret。
8. 提供数据库迁移、启动、日志、健康检查、更新、备份与回滚命令。
约束：
- 不猜测缺失的业务信息；会改变架构时先提出问题。
- 保留现有可用配置，修改前说明原因；不要提交.env、密码或密钥。
- 执行docker compose config，并检查构建命令与服务间连接。
- 最后汇总文件路径、运行步骤、验证方法、风险和待确认项。
```

数据库的位置也要提前确定：Docker 里的 MySQL 适合小型项目，重要的生产数据优先使用云数据库。

小结

Docker 的工作链路是 Dockerfile 构建出镜像，镜像再启动为容器。容器的数据存放在数据卷里，容器之间通过网络通信。编写 Dockerfile 时，把不常变化的依赖安装放在源码复制之前，构建缓存才能发挥作用。端口映射的左侧是宿主机，右侧是容器。宿主机一侧绑定 `127.0.0.1`，端口就不会暴露到公网，容器里的服务则要监听 `0.0.0.0`。同一 Compose 项目里的服务用服务名互访，`localhost` 只指向容器自己。`docker compose down -v` 会连同数据卷一起删除，执行前必须确认。

08
域名与反向代理及 SSL 证书

用户通过域名访问应用，要依次经过三个环节：DNS 把域名解析到服务器，Nginx 按路径把请求分给前端或后端，SSL 证书让浏览器与服务器之间建立可信的 HTTPS 连接。

8.1 A 记录与 CNAME 记录

域名解析靠 DNS 记录完成。A 记录把域名指向一个 IPv4 地址，CNAME 记录把域名指向另一个主机名。常用的四种记录：

类型	示例	适用
A	`@` 指向 `203.0.113.10`	根域名指向 IPv4 服务器
AAAA	`@` 指向 IPv6 地址	使用 IPv6 时
CNAME	`www` 指向 `example.com`	子域名别名或第三方服务
TXT	验证字符串	域名验证，邮件安全，ACME DNS-01

记录里的 `@` 通常代表根域名，`www` 代表 `www.example.com`。修改记录后存在缓存与 TTL，不同递归 DNS 的生效时间可能不同，改完记录不一定立刻在所有地方生效。

8.2 用 dig 验证 DNS

改完记录后，用命令确认解析结果，并区分 DNS 问题与 Nginx 站点配置问题：

```
dig +short app.example.com
dig app.example.com A
nslookup app.example.com

curl -I http://203.0.113.10 \
  -H "Host: app.example.com"
```

`dig +short app.example.com` 只输出解析得到的地址。`dig app.example.com A` 查询这个域名的 A 记录，并显示完整的查询信息。`nslookup app.example.com` 是另一种查询解析结果的工具。使用 CDN 或代理 DNS 时，查询结果通常指向代理节点，而不是服务器自己的 IP。

最后一条命令绕过 DNS，直接访问服务器 IP，并用 `-H "Host: app.example.com"` 带上 Host Header，`-I` 表示只取响应头。Nginx 按 Host Header 匹配站点，因此这条命令能把两类问题分开。直接访问 IP 能得到正确的站点，说明 Nginx 站点配置正常，问题在 DNS。直接访问 IP 也得不到正确的站点，就要检查 Nginx 的站点匹配。

8.3 单个域名承载前后端

一个域名可以同时承载前端与后端，推荐的做法是同域加路径分流：`/` 访问前端，`/api` 访问后端。三种方案对比：

方案	示例	特点
同域路径	`app.example.com/` 与 `/api`	Cookie 与 CORS 处理简单，默认采用
不同子域	`app.example.com` 与 `api.example.com`	边界清晰，需要配置 CORS 与 Cookie
业务端口	`example.com:8000`	不适合作为正式用户入口

同域方案里前端与后端同源，浏览器不对接口请求做跨域限制。不同子域方案边界清晰，但要额外配置 CORS 与 Cookie。直接暴露业务端口的方案不适合作为正式的用户入口。统一入口的原则是用户只访问 443 端口，应用的真实端口留在内部。

8.4 代理头与流式输出

请求经过 Nginx 转发后，后端看到的连接来自 Nginx。后端生成重定向 URL 或记录审计日志时，需要知道外部请求的协议，域名与客户端 IP，这些信息由 Nginx 写进代理头交给后端：

```
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

`Host` 传递用户访问的域名。`X-Real-IP` 传递客户端 IP。`X-Forwarded-For` 在已有的转发记录后面追加客户端地址。`X-Forwarded-Proto` 传递外部使用的协议，例如 HTTPS。FastAPI 位于可信代理之后时，启动后端要加上 `--proxy-headers`，后端才会读取这些头。只信任可信反向代理写入的 `X-Forwarded-For`，因为客户端也可以伪造这个头。

AI 应用的回答常以流式方式逐段输出，WebSocket 与 SSE 需要额外的代理配置：

```
# WebSocket
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";

# Server-Sent Events / streaming
proxy_buffering off;
proxy_read_timeout 3600s;
```

WebSocket 部分让 Nginx 用 1.1 版本的 HTTP 协议连接后端，并把 `Upgrade` 与 `Connection` 头转给后端，完成连接升级。SSE 部分用 `proxy_buffering off` 关闭代理缓冲。普通 JSON 接口可以使用默认缓冲，流式响应则需要尽快转发给客户端。`proxy_read_timeout 3600s` 把读取超时设为 3600 秒。超时过短，长回答会中途断开。超时过长，也要配合连接与资源限制。配置完成后，在浏览器的 Network 面板里确认数据在持续到达。

8.5 ACME 签发与验证方式

HTTPS 需要证书。“Let's Encrypt”通过 ACME 自动签发证书，流程分为验证域名控制权，签发证书与定期续期：

1. ACME Client 向 CA 请求验证挑战。
2. 客户端通过 HTTP 文件或 DNS 记录证明自己控制这个域名。
3. CA 从网络侧验证挑战。
4. 客户端生成 CSR，CA 据此签发证书。
5. 客户端自动安装证书，并定期续期。

证明域名控制权有 HTTP-01 与 DNS-01 两种方式：

验证方式	要求	适用
HTTP-01	域名已指向服务器，80 端口可访问	单域名，普通网站，配置简单
DNS-01	能通过 DNS 添加 TXT 记录或调用 API	泛域名证书，80 不可达，集中证书管理

HTTP-01 让 CA 通过 80 端口访问服务器上的验证文件，只有单个域名时优先选用。HTTP-01 申请失败时，依次检查这四项：

- DNS 解析是否已经指向服务器。
- 80 端口能否从外部访问。
- 防火墙是否放行。
- Nginx 站点是否匹配这个域名。

DNS-01 让 CA 查询域名下的 TXT 记录，适合泛域名证书或 80 端口不可达的场景。自动添加记录要用 DNS 的 API Token，这个 Token 使用最小权限，并像生产密钥一样保护。

8.6 宝塔申请证书与 HTTPS 跳转

宝塔在站点设置里完成证书的申请，安装与续签，操作分四步：

1. 确认域名已经正确解析到服务器。
2. 进入站点的 SSL 页面申请证书。
3. 选择免费证书，或上传已有证书。
4. 开启强制 HTTPS 与到期提醒。

图中是宝塔站点修改窗口的 SSL 页。几个标签对应不同的证书来源，当前证书页可以粘贴已有证书的密钥与证书内容，顶部提示条上有申请证书的按钮。



图 15　宝塔站点设置里的 SSL 页面。来源：宝塔官方文档。

开启强制 HTTPS 后，80 端口仍要保留，用于证书验证与重定向。对应的 Nginx 配置：

```
server {
    listen 80;
    server_name app.example.com;

    location /.well-known/acme-challenge/ {
        root /www/server/panel/vhost/certbot;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}
```

这个 `server` 块监听 80 端口，服务 `app.example.com`。`/.well-known/acme-challenge/` 下是 HTTP-01 的验证文件，从 `/www/server/panel/vhost/certbot` 目录读取，续签时的验证请求能够正常通过。其余请求由 `return 301` 永久重定向到 HTTPS，`$host$request_uri` 保留原来的域名与路径。宝塔通常会自动生成这类配置，不必手写路径。

强制 HTTPS 后，前端发出的所有请求都要使用安全协议，包括 API 请求，图片与字体，以及 WebSocket 连接。

8.7　Mixed Content 与 CORS 排错

Mixed Content 指 HTTPS 页面里夹带了 HTTP 请求，浏览器会拦截这类请求。典型的错误是页面地址为 `https://app.example.com`，前端仍然请求 `http://203.0.113.10:8000/api`。浏览器会阻止其中不安全的请求，例如 Fetch 与 XHR 发出的接口请求，以及加载脚本和样式表的请求。页面因此无

法获取数据或样式。修复的办法是让页面发出的所有请求都使用安全协议。前端改用相对地址 `/api`，可以同时避免协议，域名与环境的差异。

CORS 是浏览器对跨 Origin 请求的约束。不同场景的处理方式：

场景	处理
同域 `/api`	同源请求，配置简单
从 `app.example.com` 请求 `api.example.com`	后端明确允许前端 Origin
带 Cookie 或 Authorization	不能用任意 `*`，需精确 Origin 与 Credentials 配置
Postman 与 curl 请求成功，浏览器请求失败	优先检查 CORS 与 Preflight

CORS 由浏览器执行，Postman 与 curl 不受这项约束，所以会出现工具请求成功而浏览器请求失败的情况。这时优先检查 CORS 与 Preflight。CORS 也不能替代后端鉴权，攻击者可以绕开浏览器，直接请求 API。

8.8 上线验证与证书续签

上线后用这组命令检查跳转，证书与接口：

```
curl -I http://app.example.com
curl -I https://app.example.com
curl -i https://app.example.com/api/health

openssl s_client \
  -connect app.example.com:443 \
  -servername app.example.com </dev/null 2>/dev/null |
openssl x509 -noout -subject -issuer -dates
```

第一条命令取 HTTP 地址的响应头，应看到跳转到 HTTPS。第二条命令取 HTTPS 地址的响应头，确认 HTTPS 正常，安全头也在这份响应头里检查。第三条命令请求健康检查接口，`-i` 输出响应头与响应体，API 应返回预期的 JSON。最后一组命令用 `openssl s_client` 连接 443 端口取回证书。`-servername` 指定要取哪个域名的证书，`</dev/null 2>/dev/null` 让命令不等待输入并隐藏多余的输出。取回的证书交给 `openssl x509`，打印证书的主体，签发者与有效期，用来核对证书域名与有效期。命令检查通过后，再到浏览器控制台确认没有 Mixed Content 与 CORS 报错，证书链完整，资源没有 404。

免费证书的有效期通常较短，依赖自动续签，同时要开启到期提醒。续签失败的常见原因有四种：

- DNS 已经切换。
- 80 端口被关闭。
- 验证路径被代理。
- DNS Token 失效。

运维上要持续监控自动续签与证书到期时间。

HSTS 由响应头 `Strict-Transport-Security` 开启，要求浏览器只用 HTTPS 访问站点。HSTS 配错会让浏览器拒绝回退到 HTTP，证书出错时用户也无法绕过，所以要在确认 HTTPS 长期稳定之后再考虑开启。

小结

域名解析决定请求能否到达服务器：A 记录指向 IP，CNAME 指向另一个主机名，`dig` 与带 Host Header 的 `curl` 分别验证 DNS 与 Nginx 站点。推荐把前后端放在同一个域名下按路径分流，对外只开放 443 端口。Nginx 通过代理头把外部请求的信息传给后端，流式输出需要关闭缓冲并延长读取超时。证书通过 ACME 自动签发与续期，80 端口要保留给验证与跳转。HTTPS 页面混入 HTTP 请求会被拦截，跨域限制只约束浏览器，不能代替后端鉴权。

09
运维排错回滚与课堂实战

应用上线后，日常工作转向运维，核心是三件事：出故障时能定位，数据丢失时能恢复，发布出错时能回滚。最后的课堂综合实战把全天内容串成一次完整的交付。

9.1 故障分层定位与日志

线上故障按层定位，从用户入口逐层查到数据层：从 DNS 与 TLS 开始，经过 Nginx 与前端，再到 API 与数据库。每一层都用独立的工具验证，找到“第一次失败”的位置。页面报 500 时，同样从首次失败的那一层开始处理。

同一个请求会在各层留下日志，用 Request ID 把各层日志关联起来，就能追踪一个请求的完整路径。各层日志的关注点不同：

日志层	关注字段	常见问题
Nginx Access	Host，Path，Status，耗时	请求是否到达，哪个环节慢
Nginx Error	upstream，connect，time-out	502，证书，配置
应用日志	Request ID，用户，异常栈	业务错误，第三方 API
Docker 或进程	Exit Code，Restart Count	启动失败，崩溃循环
MySQL	连接，慢查询，磁盘	密码，迁移，性能

记录日志也有纪律：不记录密码与 Token，不记录完整的身份证件，也不记录模型密钥。生产环境的错误信息要能指导行动，看到日志就知道下一步怎么处理。

9.2 常见错误码排查

502 Bad Gateway 表示 Nginx 到上游应用的连接失败。排查按四个问题依次进行：

```
# 1. Is the process/container running?
docker compose ps
docker compose logs --tail 200 api

# 2. Is the port listening?
ss -lntp | grep 8000

# 3. Can the host/proxy reach it?
curl -i http://127.0.0.1:8000/health

# 4. Is Nginx config valid?
nginx -t
```

第一步确认进程或容器在运行，`docker compose ps` 查看服务状态，`docker compose logs --tail 200 api` 查看 api 服务最近 200 行日志。第二步确认端口在监听，`ss -lntp` 列出正在监听的端口与对应进程，`grep 8000` 从中筛出 8000 端口。第三步确认本机能访问应用，用 `curl` 请求健康检查接口。第四步用 `nginx -t` 检查 Nginx 配置是否有效。

判断的结论是：本机 `curl` 都失败，先修复应用。本机成功但域名访问失败，再检查 Nginx 与防火墙。

404 的来源可能完全不同，先看响应内容和 Server Header，判断响应来自 Nginx 还是后端。常见的四种现象：

现象	可能来源	修复方向
刷新前端子路由 404	Nginx 找不到文件	SPA 的 `try_files`
`/api` 返回前端 HTML	路由顺序错误或回退规则误匹配 API	独立的 `location /api/`
FastAPI JSON 404	路径被代理改写错误	检查尾斜杠与 root path
静态 JS 与 CSS 404	构建 base path 错误	检查资源前缀与站点根目录

`/api` 返回前端 HTML 时，接口请求被 SPA 的回退规则当成前端路由，返回的是 `index.html`，要为 `/api/` 单独写一个 `location`。返回 FastAPI 的 JSON 404 时，请求已经到达后端，问题出在代理改写后的路径，要检查 `proxy_pass` 的尾斜杠与 root path。静态 JS 与 CSS 返回 404，通常是构建时设置的 base path 与站点根目录不一致。

9.3 数据库连接与资源不足

数据库连接失败时，先查网络与账号，再查 Schema 与迁移，逐项检查：

1. 检查主机名，容器内用 `db`，宿主机进程可能用 `127.0.0.1`。

2. 检查端口是否正确，数据库是否真正 Ready。
3. 核对用户，密码与数据库名，确认与初始化配置一致。
4. 确认用户有目标数据库的权限，字符集符合项目要求。
5. 确认当前代码需要的迁移已经执行。

容器里有一个常见陷阱：API 容器里的 `localhost:3306` 指向 API 容器自身，无法连接到数据库容器。

服务器资源不足时，构建时与运行时的瓶颈不同。查看资源用五条命令：

```
free -h
df -h
du -sh /var/lib/docker/*
docker stats
top
```

`free -h` 查看内存使用情况，`-h` 让输出使用易读的单位。`df -h` 查看各磁盘分区的使用率。`du -sh /var/lib/docker/*` 统计 Docker 数据目录下各子目录占用的空间。`docker stats` 实时显示各容器的资源占用。`top` 实时显示系统中各进程的资源占用。

Node 前端构建时常出现内存峰值。MySQL，模型服务与多个 Worker 会长期占用内存。占用磁盘的主要是 Docker 镜像与构建缓存，以及日志和数据库。内存不足触发 OOM Kill 时，应用看起来像“突然消失”，要查看系统日志和容器退出码。

9.4 备份恢复与安全基线

备份覆盖四类对象，每一类都要验证能够恢复：

对象	备份方式	恢复验证
MySQL	定时逻辑备份加云快照	导入新库并执行关键查询
上传文件	Volume，目录同步，对象存储版本	随机抽样文件能够打开
Compose 与 Nginx 配置	纳入 Git	新服务器可以复现
Env 变量名	`.env.example` 加 Secret 清单	新环境的变量齐全
证书	以自动重新签发为主	续签任务与域名验证可用

重要数据遵循“3-2-1”：至少 3 份，存放在 2 种介质上，其中 1 份放在异地。练习项目也要养成这个习惯。

单机部署的**安全基线**是默认拒绝，按需开放，具体有十项：

- SSH 使用密钥，限制 root 登录。
- 面板入口限制来源并启用 HTTPS。

- 公网只开放 22，80 与 443。
- MySQL，Redis 与 API 端口不暴露公网。
- 运行进程使用非 root 用户。
- 系统与依赖定期更新。
- Secret 最小权限并定期轮换。
- 数据库与上传文件定期备份。
- 日志与磁盘空间设置告警。
- 删除离职成员与过期的 Deploy Key。

一台服务器适合小型项目和教学。业务增长后，再把数据库，对象存储与负载均衡拆分出去。

9.5 规范发布与回滚条件

一次规范的发布分七个阶段，发布前，发布中与发布后都要留下证据：

阶段	动作
准备	分支已合并，测试通过，记录版本，备份，确认回滚点
构建	锁文件安装，生产构建，镜像打 Tag，扫描敏感信息
迁移	执行兼容性数据库迁移，失败则停止
发布	重启或替换进程与容器，保持变更范围最小
验证	Health，首页，登录，数据库，关键 API，日志
观察	错误率，响应时间，资源，用户反馈
收尾	记录结果，删除临时权限，更新变更日志

回滚指把服务恢复到上一个可用的版本。回滚的触发条件要提前定义，包括阈值与决策人。出现这五种情况之一，就应当回滚：

- 健康检查持续失败，服务无法启动。
- 登录，支付，核心对话等关键链路不可用。
- 错误率或响应时间明显超过基线。
- 数据写入异常，权限越权，Secret 泄漏等安全事件。
- 修复时间不可控，继续在线排查会扩大影响。

决策原则是先恢复服务，再分析根因。回滚操作同样需要验证与记录。

9.6 手工部署与 CI/CD

先把手工部署的 SOP 做对，再将流程自动化。自动化把已经验证过的 SOP 固化为可以重复执行的流程，各环节依次是：

环节	内容
Push 或合并	触发后续环节
CI	执行 Lint 与 Test
Build	构建 Image，并用 SHA 标记版本
Registry	存放构建好的镜像
Deploy	在服务器上 Pull 镜像并 Up 启动
Verify	做 Smoke 验证

CI 使用与本地一致的 pnpm 与 uv 版本，以及相同的锁文件策略。

9.7 课堂综合实战

课堂综合实战的目标是把 Day 3 与 Day 4 的项目真正交付给另一个人访问，按七步完成：

1. 确认上线前修复已经合并到主分支。
2. 购买服务器后，完成宝塔基础环境配置。
3. 选择宝塔手工路线或 Docker Compose 路线部署。
4. 配置域名，`/api` 反向代理与 SSL。
5. 另一组使用手机网络访问，并完成关键流程测试。
6. 主动制造一次错误端口，按日志定位并恢复。
7. 更新一个版本，再回滚到上一个版本。

第 1 步对应 Git 协作，第 2 到第 4 步完成部署，最后三步分别检验外部访问，排错与回滚。

9.8 官方参考资料

官方参考资料分两组。协作，Docker 与本地模型：

- Pro Git Book：Git 基础与分支
- GitHub Docs：GitHub Flow
- Docker Docs：Compose
- pnpm Docs：Working with Docker
- uv Docs：Using uv in Docker

- Unsloth Docs：Qwen3.6 本地推理教程
- Unsloth Docs：Unsloth Studio
- llama.cpp：本地推理与 OpenAI 兼容服务

宝塔，网络与 HTTPS：

- 宝塔面板官方文档
- 宝塔：反向代理
- 宝塔：部署 SSL 证书
- 宝塔：Node.js PM2 部署
- NGINX Docs：Reverse Proxy
- Cloudflare Docs：DNS
- “Let's Encrypt”：How It Works
- MDN：Mixed Content
- 工业和信息化部：互联网信息服务备案相关规定

小结

排查线上故障要逐层验证，从第一次失败的那一层入手，并用 Request ID 关联各层日志。502 先查应用进程，最后查 Nginx 配置，404 先分清响应来自 Nginx 还是后端。数据安全依靠能够恢复的备份与默认拒绝的安全基线。发布分阶段进行并保留证据，关键链路异常时先回滚恢复服务，再分析根因。课堂综合实战用一次完整的交付，把这些环节都演练一遍。

练习与作业

复习题

1. 说明 Commit 与 Push 的区别，写出把本地新提交同步到 GitHub 的命令。
2. 说明工作区，暂存区与本地仓库各自存放的内容，以及 `git add` 与 `git commit` 分别完成的动作。
3. 比较 `git commit --amend` 与 `git revert` 适用的撤销场景，写出共享分支上禁止执行的两类命令。
4. 说明冲突标记中 `HEAD` 一侧与另一侧分别代表的内容，以及删除全部标记后必须重新运行的检查。
5. 说明公网只保留的三个端口及各自用途，以及应用端口与数据库端口的开放原则。
6. 说明 SPA 刷新子路由返回 404 的原因，写出修复所用的 Nginx 配置。
7. 写出前后端联调的五步验证顺序，说明“在哪一层第一次失败”的定位方法。

8. 说明 `-p 127.0.0.1:8000:8000` 两侧端口的含义，以及容器之间不能用 `localhost` 互访的原因。
9. 比较 Mixed Content 与 CORS 两类问题的成因与处理方法。
10. 写出 502 Bad Gateway 的四步排查顺序，说明本机 `curl` 成功而域名访问失败时的检查方向。
11. 说明回滚的触发条件与决策原则。
12. 说明本地部署 Qwen3.6 的学习路线，以及业务代码改用 `llama-server` 时需要替换的两项配置。

课后作业

课后独立完成课堂综合实战的全部七步，把 Day 3 与 Day 4 的项目交付给另一个人访问。完成后逐项核对：

1. 上线前的修复已经合并到主分支。
2. 服务器已经购买，宝塔基础环境配置完成。
3. 应用已经用宝塔手工路线或 Docker Compose 路线部署上线。
4. 域名，`/api` 反向代理与 SSL 已经配置完成。
5. 另一组已经用手机网络访问，并完成关键流程测试。
6. 主动制造过一次错误端口，并按日志定位与恢复。
7. 更新过一个版本，并回滚到上一个版本。

A
拓展：Qwen3.6 本地模型部署

拓展内容基于 Unsloth 官方推理教程，供按需选学，目标是在本机部署 Qwen3.6 模型，并对外提供与 OpenAI 兼容的接口。建议的学习路线是先在 Unsloth Studio 里完成部署与对话测试，再转向 llama.cpp 的命令行服务。

A.1　路线选择与内存需求

本地运行 Qwen3.6 有两条路线。Unsloth Studio 是可视化路线，适合教学与入门。llama.cpp 是命令行路线，适合集成到其他程序。学习时先选 Studio，运行成功后再决定是否转向 llama.cpp。

模型以 `4-bit` 量化运行时需要的总内存：

模型	总内存
Qwen3.6-27B	约 18 GB
35B-A3B	约 23 GB

总内存指 RAM 与 VRAM 之和，应高于量化文件的大小。模型的上下文最大 256K，练习时先从 16K 起步。

A.2 安装并启动 Studio

Studio 安装后在浏览器里使用，先完成从安装到对话的最小完整流程。安装与启动的命令：

```
# macOS / Linux / WSL
curl -fsSL https://unsloth.ai/install.sh | sh

# 只监听本机
unsloth studio -H 127.0.0.1 -p 8888

# 临时安全远程访问
unsloth studio --secure
```

在 macOS，Linux 或 WSL 上，第一条命令下载官方安装脚本，交给 `sh` 执行安装。Windows 的 PowerShell 使用官方的 `install.ps1` 安装。

`unsloth studio -H 127.0.0.1 -p 8888` 启动 Studio，`-H` 指定监听地址，`-p` 指定端口，只监听 `127.0.0.1` 时只有本机能访问。启动后，在浏览器打开 `127.0.0.1:8888` 即可使用。首次启动先设置 Studio 的登录密码。

需要从别的机器临时访问时，使用 `unsloth studio --secure` 开启临时的安全远程访问。

A.3 模型搜索与量化选择

在 Studio 里搜索 `Qwen3.6`，选择 Dynamic `4-bit` 量化版本并下载。默认使用 `UD-Q4_K_XL`，一般的学习环境优先选 `27B` 加 `UD-Q4_K_XL`，资源更容易控制。需要加速时，再尝试 MTP 版本。MTP 版本通常要多预留约 1 GB 的 RAM 或 VRAM，并要实测草稿 Token 数。

图中是 Studio 里的两次模型搜索，结果都来自 Hugging Face。搜索 `qwen3.6` 并展开 `35B-A3B` 的 GGUF 版本，每个量化版本都标出了文件大小，`UD-Q4_K_XL` 标为推荐。搜索 `qwen3.6 mtp`，MTP 版本单独列出，`27B` 与 `35B-A3B` 各有一个。

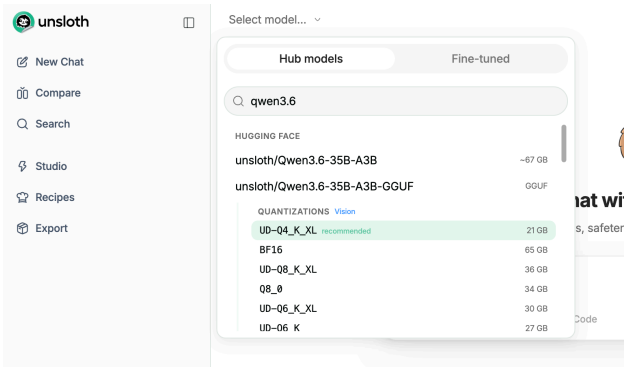


图 16　搜索 Qwen3.6 得到的量化版本。来源：Unsloth Docs。

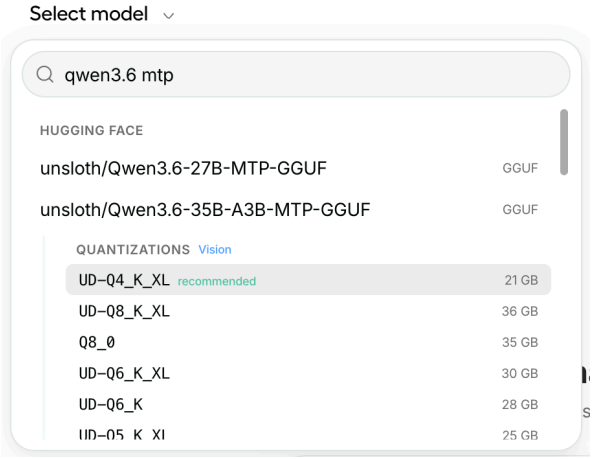


图 17　搜索 MTP 版本的结果。来源：Unsloth Docs。

A.4　推理模式与参数

运行前先对齐推理模式与参数。Qwen3.6 有思考模式 Thinking 与非思考模式 Non-thinking，思考模式又分通用与精确编码两种用法，三种情况使用不同的参数：

模式	Temp	Top-p
Thinking 通用	1.0	0.95
Thinking 编码	0.6	0.95
Non-thinking	0.7	0.8

Temp 是温度，数值越高，生成结果越多样。Top-p 与 top-k 限定每一步从哪些候选 Token 中采样。三种模式都使用 `top-k=20`。非思考模式另外设置 `presence penalty=1.5`，用来减少重复的内容。

A.5　Studio 中的功能验证

Studio 不只是聊天框，同一界面还能验证搜索与工具调用，代码执行，以及持续对话。验证时注意三点：工具调用要确认是否成功，不能只看最终答案。记录首 Token 时间，生成速度与内存占用。需要连续推理时，再开启 Preserve Thinking。

图中模型先调用工具读取网页，界面上标出了所用的工具，再根据读到的内容回答。输入框下方有 Preserve Thinking 开关。

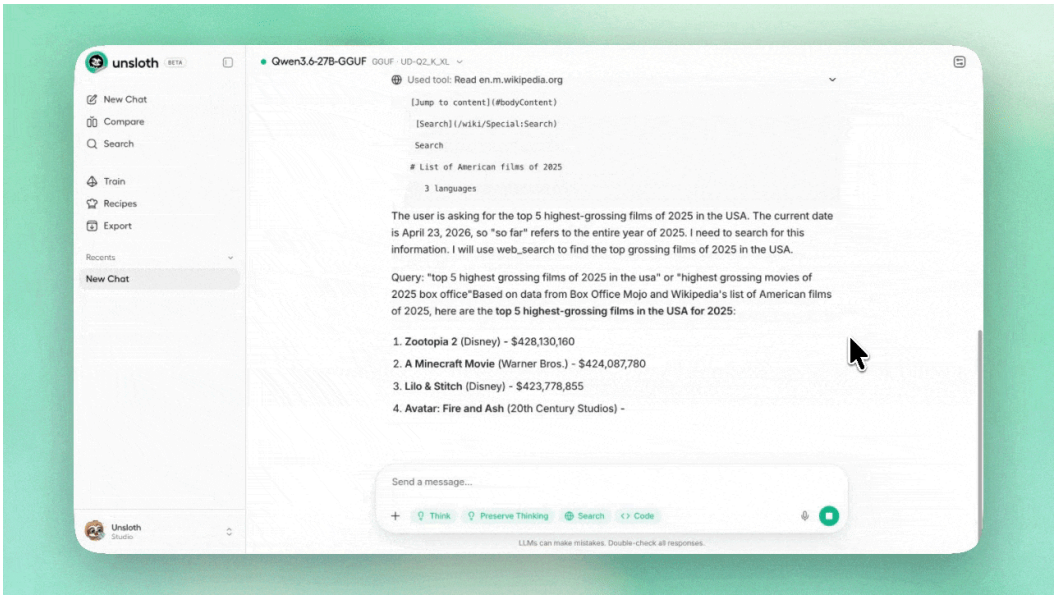


图 18　模型在 Studio 中调用工具后回答。来源：Unsloth Docs。

A.6　llama.cpp 命令行路线

Studio 运行成功后，可以用 llama.cpp 转到能写成脚本的命令行路线。最新版 llama.cpp 可以直接从 Hugging Face 拉取 GGUF 格式的模型。

```
git clone https://github.com/ggml-org/llama.cpp
cmake llama.cpp -B llama.cpp/build \
  -DBUILD_SHARED_LIBS=OFF -DGGML_CUDA=ON
cmake --build llama.cpp/build --config Release -j \
  --target llama-cli llama-server

./llama.cpp/build/bin/llama-cli \
  -hf unsloth/Qwen3.6-27B-GGUF:UD-Q4_K_XL \
  --temp 1.0 --top-p 0.95 --top-k 20 --min-p 0.0
```

`git clone` 下载 llama.cpp 的源码。第一条 `cmake` 生成构建配置，构建目录是 `llama.cpp/build`。`-DBUILD_SHARED_LIBS=OFF` 关闭共享库，`-DGGML_CUDA=ON` 启用 CUDA。第二条 `cmake --build` 按 Release 配置编译，`-j` 并行编译，`--target` 只编译 `llama-cli` 与 `llama-server` 两个程序。

最后用 `llama-cli` 运行模型。`-hf` 从 Hugging Face 拉取 `unsloth/Qwen3.6-27B-GGUF` 里的 `UD-Q4_K_XL` 量化版本。后面四个参数设置采样：温度 1.0 与 Top-p 0.95，以及 Top-k 20 与 Min-p 0.0。这组取值与 Thinking 通用模式的推荐值一致。

没有 NVIDIA GPU 的设备或 Apple Metal 设备，构建时改用 `-DGGML_CUDA=OFF`。精确编码任务把温度改为 0.6。模型的缓存目录可以用环境变量 `LLAMA_CACHE` 固定。CUDA 避免使用 13.2 版本，官方页面提示使用低于 13.2 的版本或 13.3。

A.7　llama-server 兼容接口

`llama-server` 把本地模型包装成 **OpenAI 兼容接口**。按 OpenAI 接口编写的业务代码，改用本地模型时只替换 Base URL 与模型名。

```
./llama.cpp/build/bin/llama-server \
  -hf unsloth/Qwen3.6-27B-GGUF:UD-Q4_K_XL \
  --alias qwen3.6-local \
  --ctx-size 16384 --port 8001 \
  --chat-template-kwargs \
  '{"enable_thinking":false}'
```

`-hf` 指定要拉取的模型。`--alias qwen3.6-local` 给模型起一个对外的名字，业务代码里的模型名就填 `qwen3.6-local`。`--ctx-size 16384` 把上下文设为 16384，对应练习时从 16K 起步。`--port 8001` 指定服务端口。`--chat-template-kwargs '{"enable_thinking":false}'` 关闭思考模式。

服务启动后，Python SDK 的 Base URL 填 `http://127.0.0.1:8001/v1`。服务默认只有本机能访问，远程调用必须增加鉴权与 HTTPS。

A.8　MTP 可选加速

MTP 让模型一次预测几个草稿 Token，再统一验证。验证通过的 Token 直接采用，生成速度因此加快。MTP 是可选的加速手段，不是固定开关。教程报告约 1.4 到 2.2 倍的生成加速。草稿 Token 数要依硬件实测：从 1 到 6 逐个测试，常见起点是 2，最终看实测的速度与内存。

图中左右两幅分别是 `27B` 与 `35B-A3B` 的测试结果。横轴是草稿 Token 数，0 表示不开 MTP，1 到 6 表示开启 MTP。纵轴是每秒生成的 Token 数。彩色细线对应不同大小的量化版本，颜色与右侧色条的文件大小对应，黑色粗线概括了各量化版本的整体走势。开启 MTP 后曲线上升，草稿 Token 数继续增大时，曲线逐渐变平。

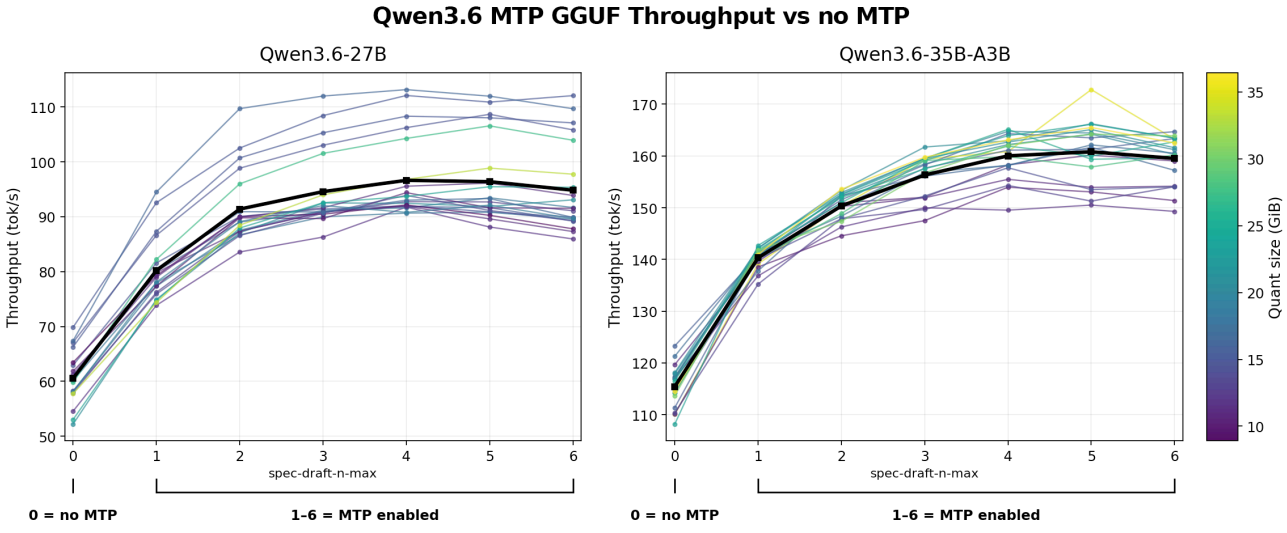


图 19　Qwen3.6 开启与不开 MTP 时的生成速度对比。来源：Unsloth Docs。

小结

本地部署 Qwen3.6 先用 Studio 的可视化界面完成部署与测试，再按需要转向可以写成脚本的 llama.cpp。选模型时先看内存，`4-bit` 量化的 `27B` 约需 18 GB，`35B-A3B` 约需 23 GB。推理参数随模式变化，思考与非思考模式的温度和采样设置各不相同。`llama-server` 对外提供 OpenAI 兼容接口，接入时只改 Base URL 与模型名，对外开放前必须加上鉴权与 HTTPS。MTP 能提升生成速度，草稿 Token 数要在自己的硬件上实测确定。